

INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 2040

SYSTEM PROGRAMMING

WEEK 7 LECTURE 2

MACHINE LEVEL PROGRAMMING II

PROCEDURES & RECURSION

Machine level Programming

- ★ Procedures
 - Stack Structure
 - Calling Conventions
 - ★ Passing control
 - ★ Passing data
 - ★ Managing local data
 - Illustration of Recursion



Mechanisms in Procedures

- ✳ Passing control
 - To beginning of procedure code
 - Back to return point
- ✳ Passing data
 - Procedure arguments
 - Return value
- ✳ Memory management
 - Allocate during procedure execution
 - Deallocate upon return
- ✳ Mechanisms all implemented with machine instructions
- ✳ x86-64 implementation of a procedure uses only those mechanisms required

```
P (...) {  
    .  
    .  
    y = Q(x);  
    print(y)  
    .  
}
```

```
int Q(int i)  
{  
    int t = 3*i;  
    int v[10];  
    .  
    .  
    return v[t];  
}
```

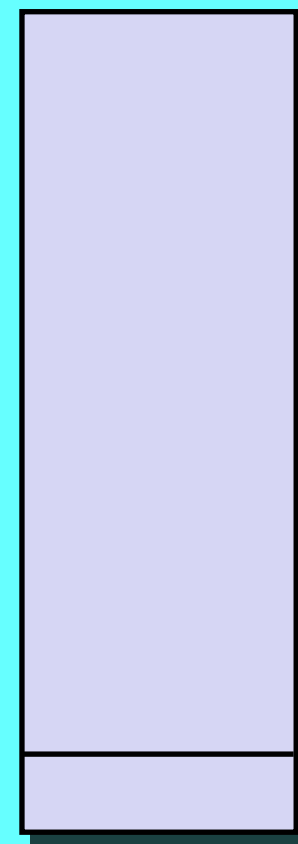
Stack Push & Pop Operations

- Stack plays a vital role in the handling of procedure calls.
- Stack is a data structure where values can be added or deleted, but only according to a “last-in, first-out” discipline.
- Add data to stack via a push operation
- Remove data from stack via a pop operation
- A stack can be implemented as an array, where we always insert and remove elements from one end of the array.
- This end is called the top of the stack.

x86-64 Stack

- ★ Region of memory managed with stack discipline
- ★ Grows toward lower addresses
- ★ Register `%rsp` contains lowest stack address
 - address of “top” element

Stack Pointer: `%rsp` →



Stack “Top”

Decreasing
Addresses

Stack
Grows
Upwards

x86-64 Stack: Push

★ **pushq Src**

- Fetch operand at *Src*
- Decrement **%rsp** by 8
- Write operand at address given by **%rsp**

Stack Pointer: %rsp → 0x8FF8

pushq %rax

%rsp ← %rsp – 8
(%rsp) ← %rax

-8
↑
0x9000

Stack “Top”

**Contents
of %rax
8 bytes**

Decreasing
Addresses

Stack
Grows
Upwards

x86-64 Stack: Pop

★ **popq Dest**

- ❖ Read value at address given by **%rsp**
- ❖ Store value at **Dest** (must be register)
- ❖ Increment **%rsp** by 8

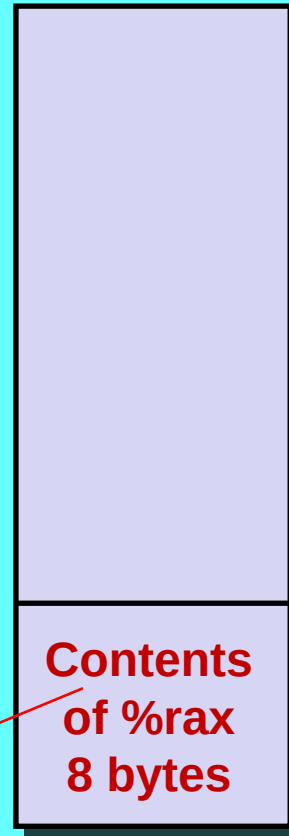
Stack Pointer: %rsp → 0x8FF8

popq %rax

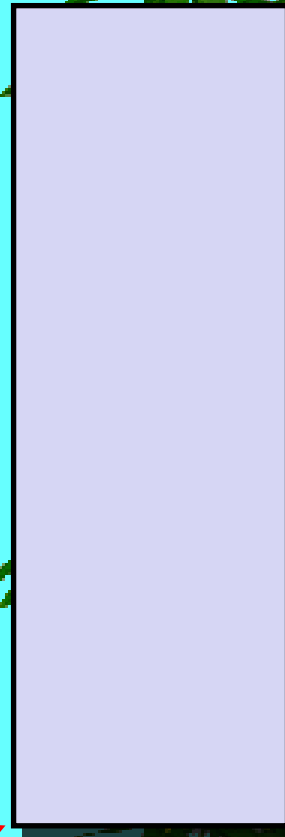
%rax ← (%rsp)

%rsp ← %rsp + 8

Before Pop



After Pop



Stack "Top"

Procedure Control Flow

- ★ Use stack to support procedure call and return
- ★ **Procedure call:** `call label`
 - **Push return address in %rip onto stack**
 - **Jump to *label***
- ★ **Return address:**
 - Address of the next instruction right after call
 - Example from disassembly
- ★ **Procedure return:** `ret`
 - **Pop address from stack into %rip**
 - **Jump to address**



Code Examples

```
void multstore
(long x, long y, long *dest)
{
    long t = mult2(x, y);
    *dest = t;
}
```

```
0000000000400540 <multstore>:
400540: push    %rbx                # Save %rbx
400541: mov     %rdx,%rbx           # Save dest
400544: callq   400550 <mult2>      # mult2(x,y)
400549: mov     %rax, (%rbx)         # Save at dest
40054c: pop     %rbx                # Restore %rbx
40054d: retq                               # Return
```

```
long mult2
(long a, long b)
{
    long s = a * b;
    return s;
}
```

```
0000000000400550 <mult2>:
400550: mov     %rdi,%rax           # a
400553: imul    %rsi,%rax           # a * b
400557: retq                               # Return
```

Control Flow Example Step #1

```
00000000000400540 <multstore>:  
.  
.  
400544: callq 400550 <mult2>  
400549: mov %rax, (%rbx)  
.  
.
```

```
00000000000400550 <mult2>:  
400550: mov %rdi, %rax  
.  
.  
400557: retq
```

0x130

0x128

0x120

%rsp

0x120

%rip

0x400544

Control Flow Example Step #2

```
00000000000400540 <multstore>:
```

•
•
•

```
400544: callq 400550 <mult2>
```

```
400549: mov    %rax, (%rbx) ←
```

•
•

```
00000000000400550 <mult2>:
```

```
400550: mov    %rdi, %rax ←
```

•
•

```
400557: retq
```

0x130

0x128

0x120

0x118

%rsp

%rip

0x400549

0x118

0x400550

Control Flow Example Step #3

00000000000400540 <multstore>:

```
•  
•  
400544: callq 400550 <mult2>  
400549: mov    %rax, (%rbx) ←  
•  
•
```

00000000000400550 <mult2>:

```
400550: mov    %rdi,%rax  
•  
•  
400557: retq ←
```

0x130

0x128

0x120

0x118

%rsp

%rip

0x400549

0x118

0x400557

Control Flow Example Step #4

```
00000000000400540 <multstore>:  
.  
.  
400544: callq    400550 <mult2>  
400549: mov     %rax, (%rbx)  
.  
.
```

```
00000000000400550 <mult2>:  
400550: mov     %rdi, %rax  
.  
.  
400557: retq
```

0x130

0x128

0x120

%rsp

0x120

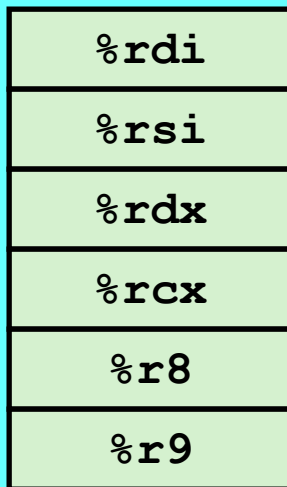
%rip

0x400549

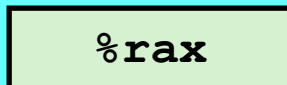
Procedure Data Flow

Registers

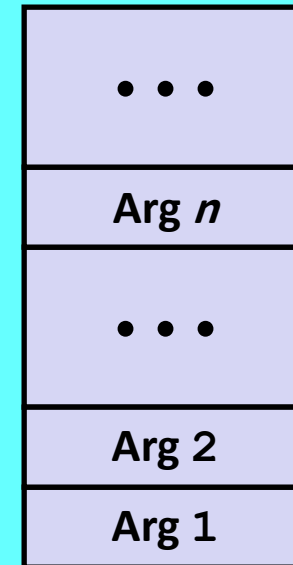
- ★ First 6 arguments



- ★ Return value



Stack



- ★ Only allocate stack space when needed

Data Flow Examples

```
void multstore
(long x, long y, long *dest)
{
    long t = mult2(x, y);
    *dest = t;
}
```

```
0000000000400540 <multstore>:
    # x in %rdi, y in %rsi, dest in %rdx
    ...
400541: mov     %rdx,%rbx        # Save dest
400544: callq   400550 <mult2>    # mult2(x,y)
    # t in %rax
400549: mov     %rax, (%rbx)      # Save at dest
    ...
```

```
long mult2
(long a, long b)
{
    long s = a * b;
    return s;
}
```

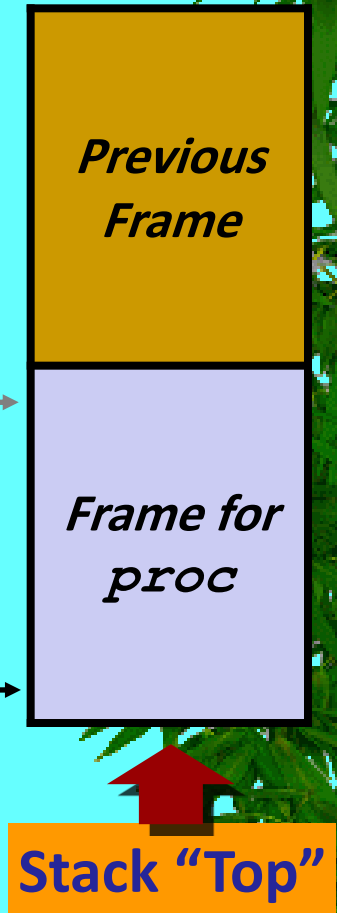
```
0000000000400550 <mult2>:
    # a in %rdi, b in %rsi
400550:  mov     %rdi,%rax        # a
400553:  imul    %rsi,%rax        # a * b
    # s in %rax
400557:  retq                      # Return
```


Stack Frames

- * Contents
 - Return information
 - Local storage (if needed)
 - Temporary space (if needed)
- * Management
 - Space allocated when enter procedure
 - * “Set-up” code
 - * Includes push by `call` instruction
 - Deallocated when return
 - * “Finish” code
 - * Includes pop by `ret` instruction

Frame Pointer: `%rbp`
(Optional)

Stack Pointer: `%rsp`



RECURSION

- * **Languages that support recursion**
 - e.g., C, Pascal, C++, Java
 - Code must be “*Reentrant*”
 - * Multiple simultaneous instantiations of single procedure
 - Need some place to store state of each instantiation
 - * Arguments
 - * Local variables
 - * Return pointer
- * **Stack discipline**
 - State for given procedure needed for limited time
 - * From when called to when return
 - Callee returns before caller does
- * **Stack allocated in *Frames***
 - state for single procedure instantiation

Call Chain Example

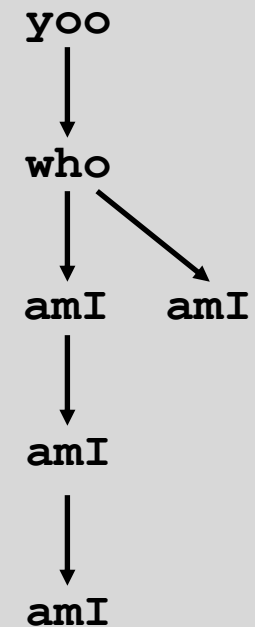
```
yoo (...)  
{  
  .  
  .  
  who ();  
  .  
  .  
}
```

```
who (...)  
{  
  . . .  
  amI ();  
  . . .  
  amI ();  
  . . .  
}
```

```
amI (...)  
{  
  .  
  .  
  amI ();  
  .  
  .  
}
```

Procedure amI () is recursive

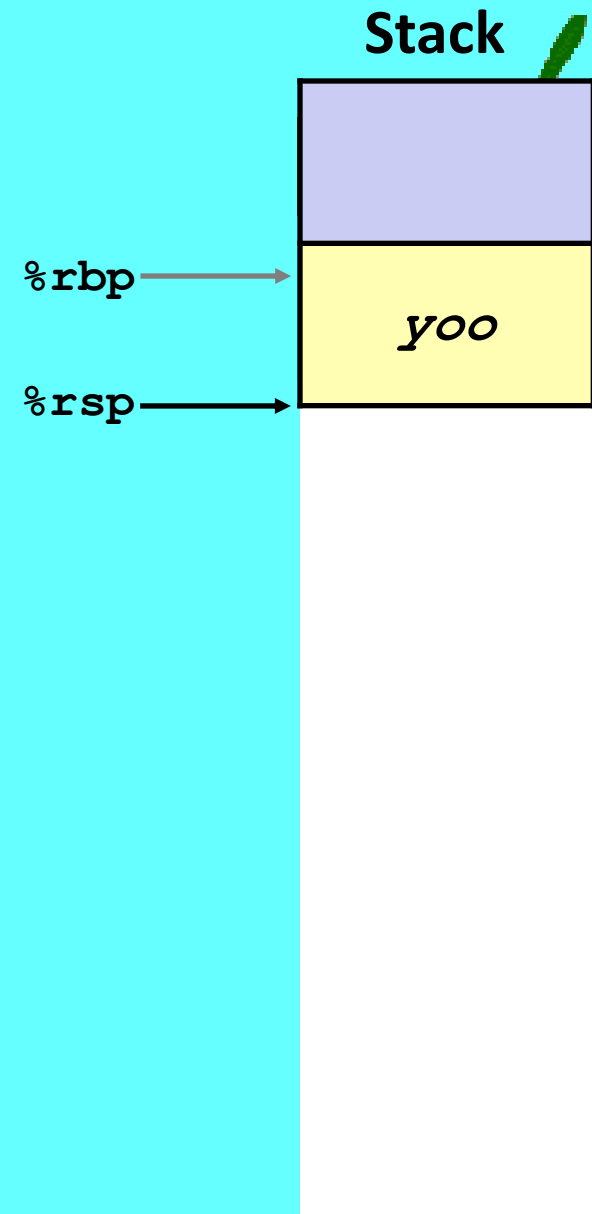
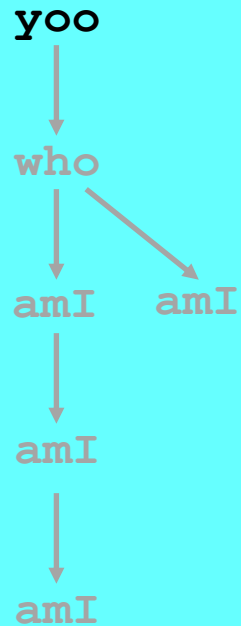
Example
Call Chain



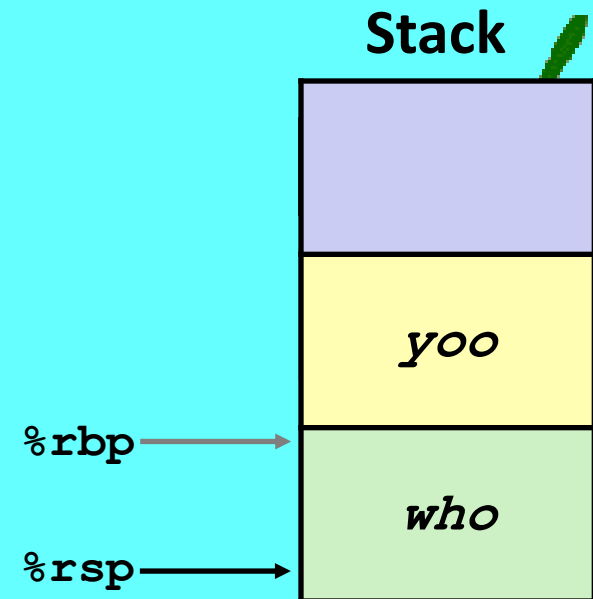
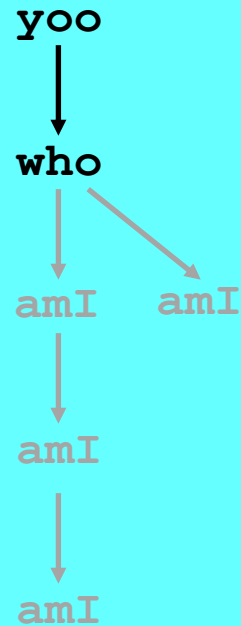
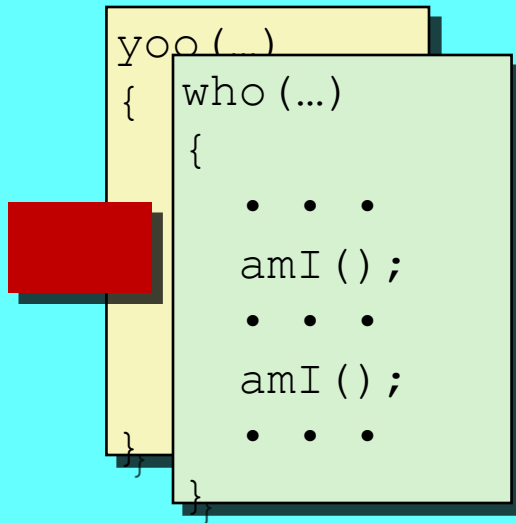
Example



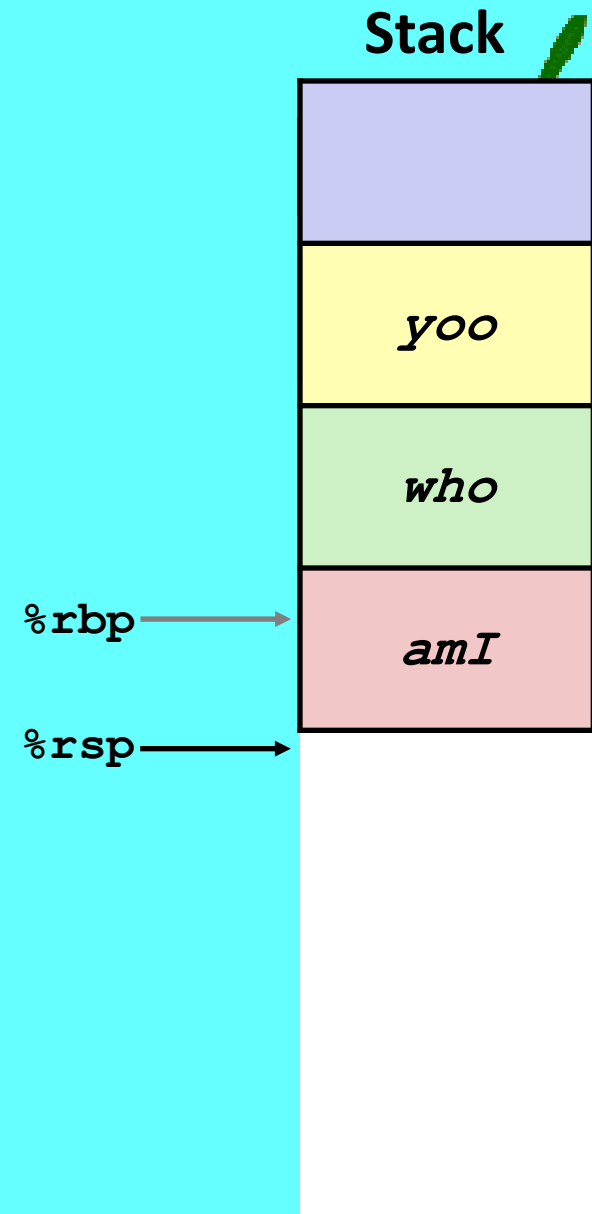
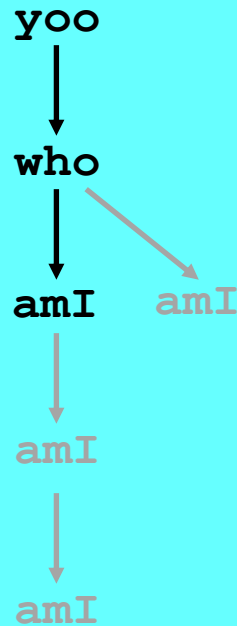
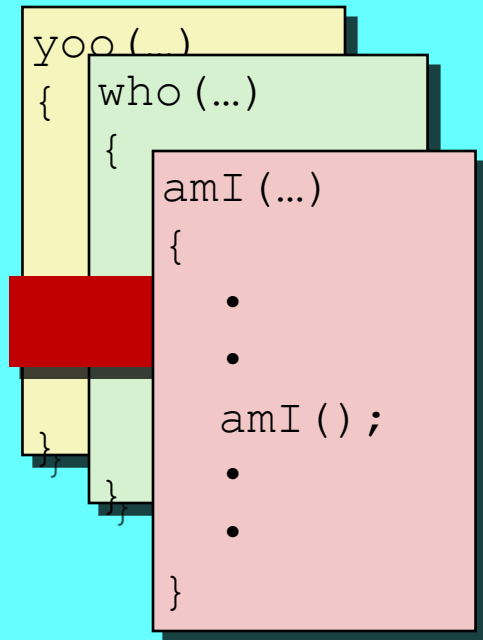
```
yoo (...)  
{  
  .  
  .  
  who ();  
  .  
  .  
}
```



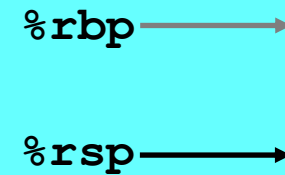
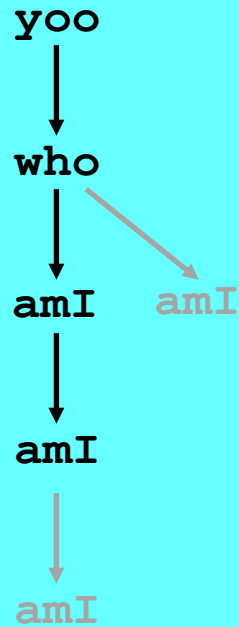
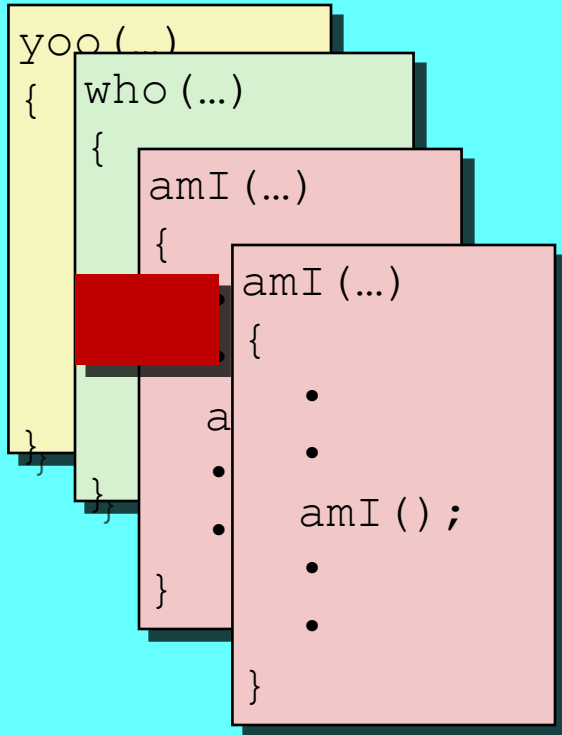
Example



Example



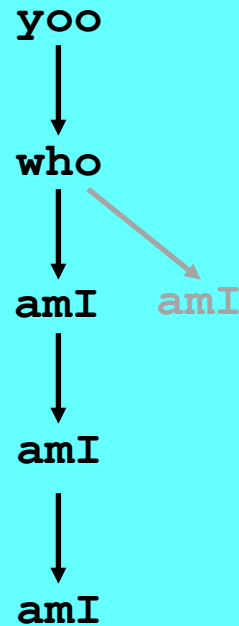
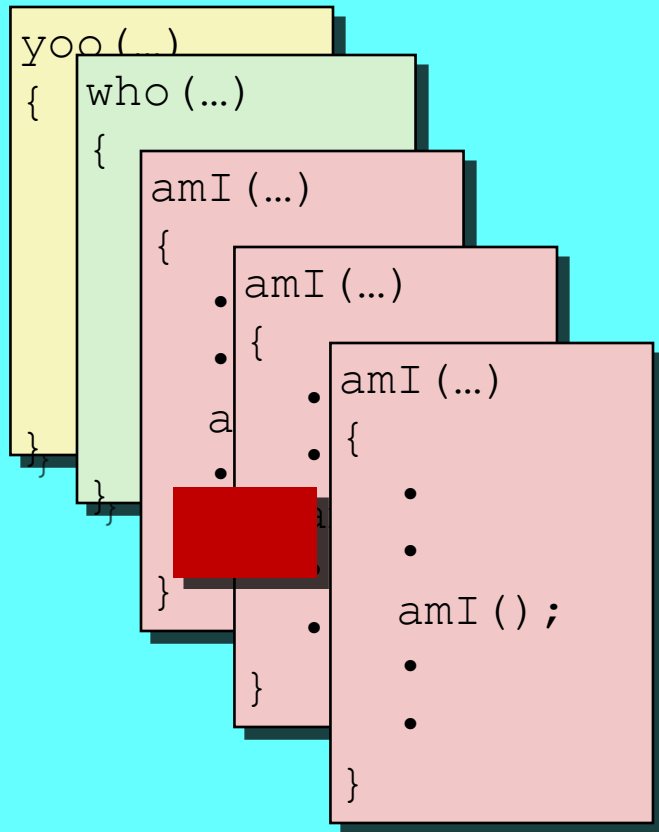
Example



Stack



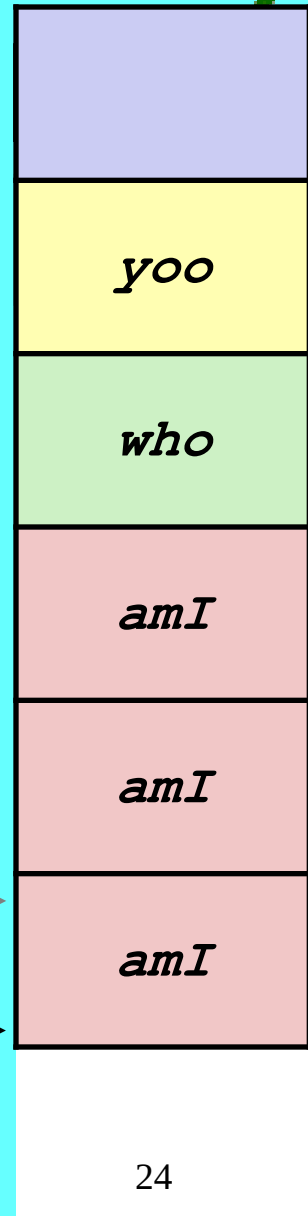
Example



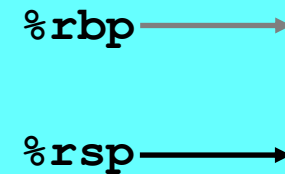
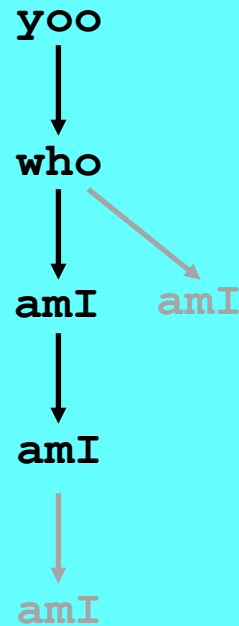
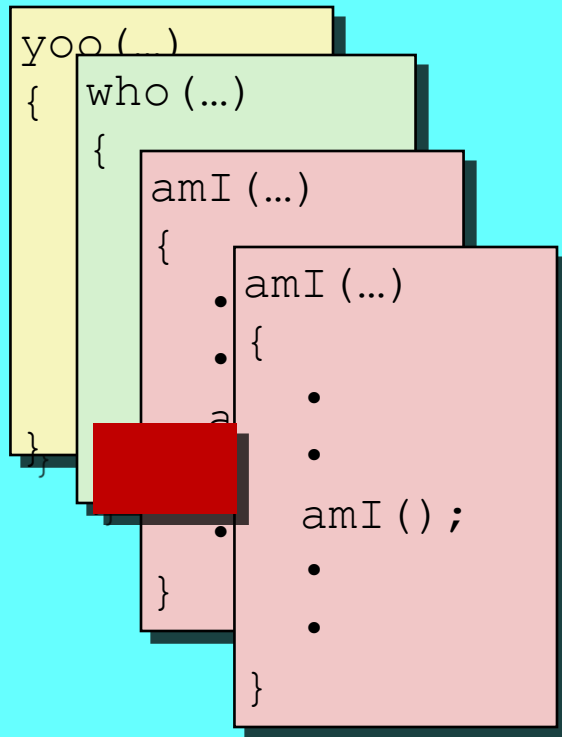
Stack

`%rbp` →

`%rsp` →



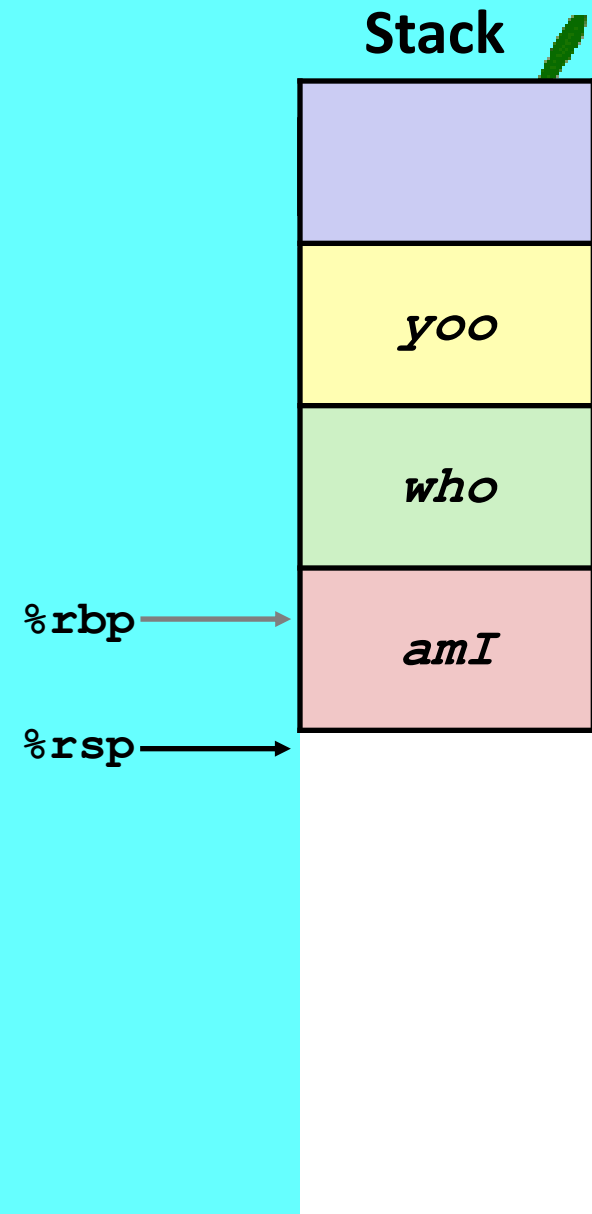
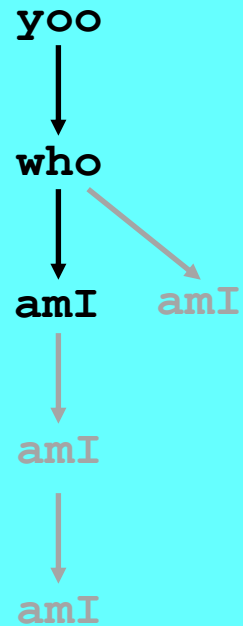
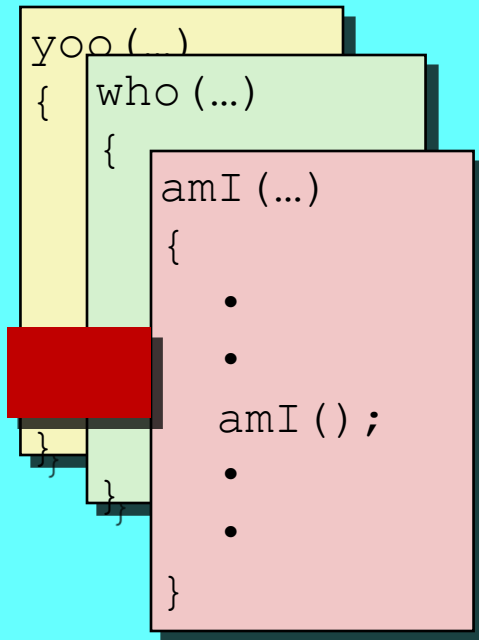
Example



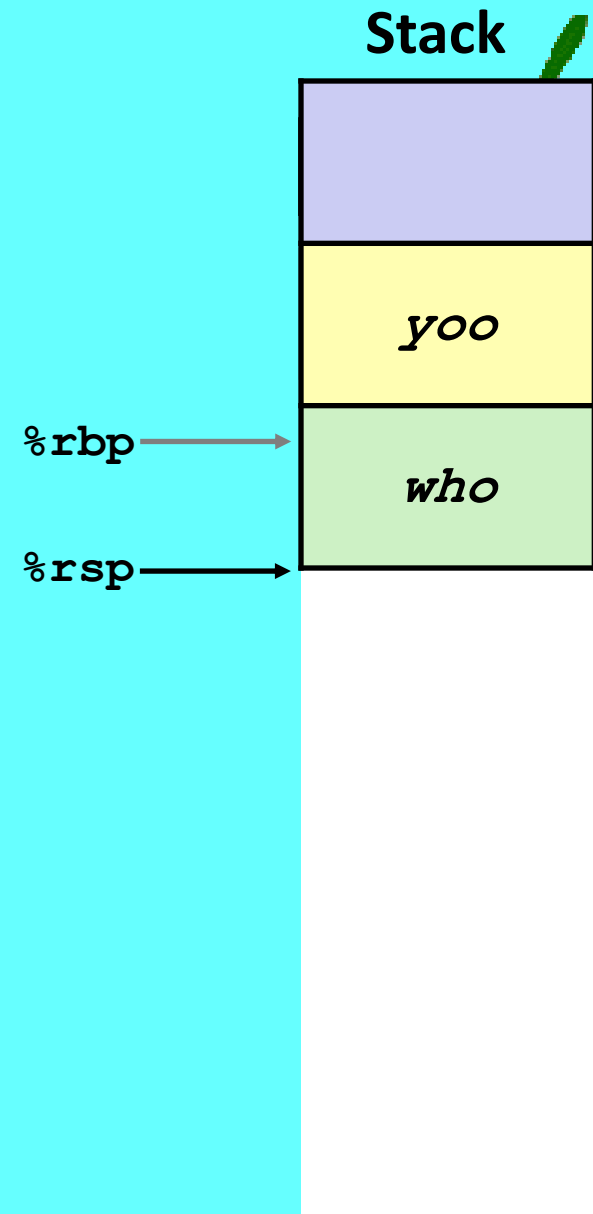
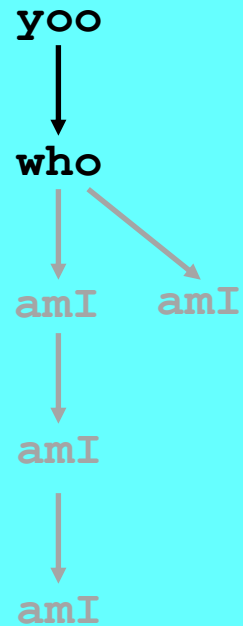
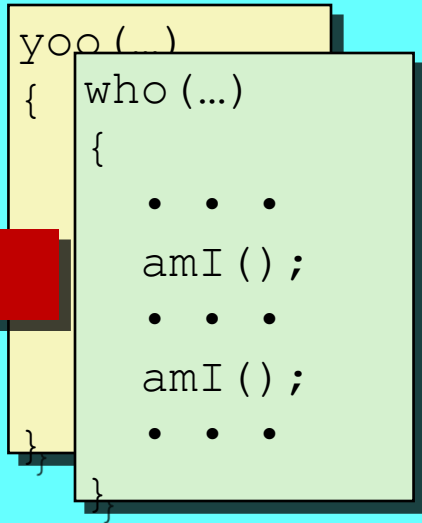
Stack



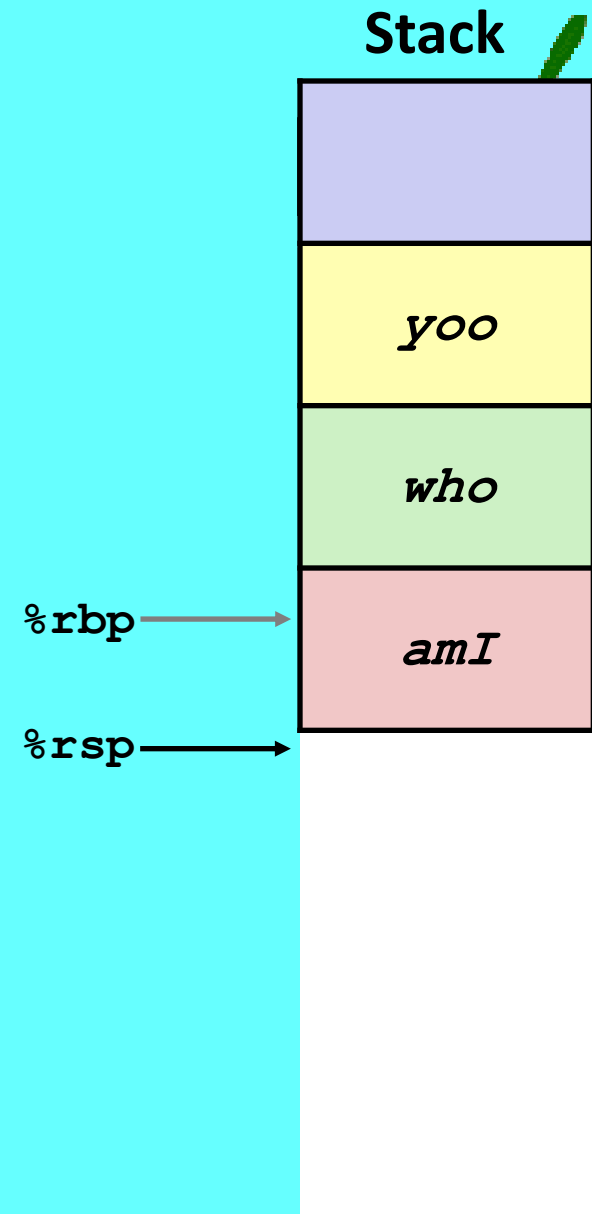
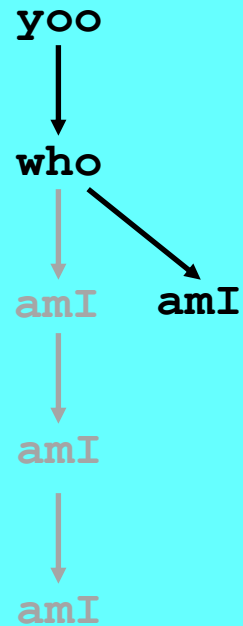
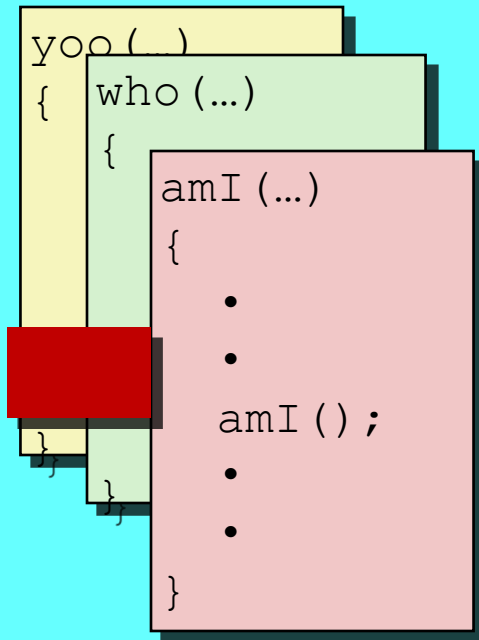
Example



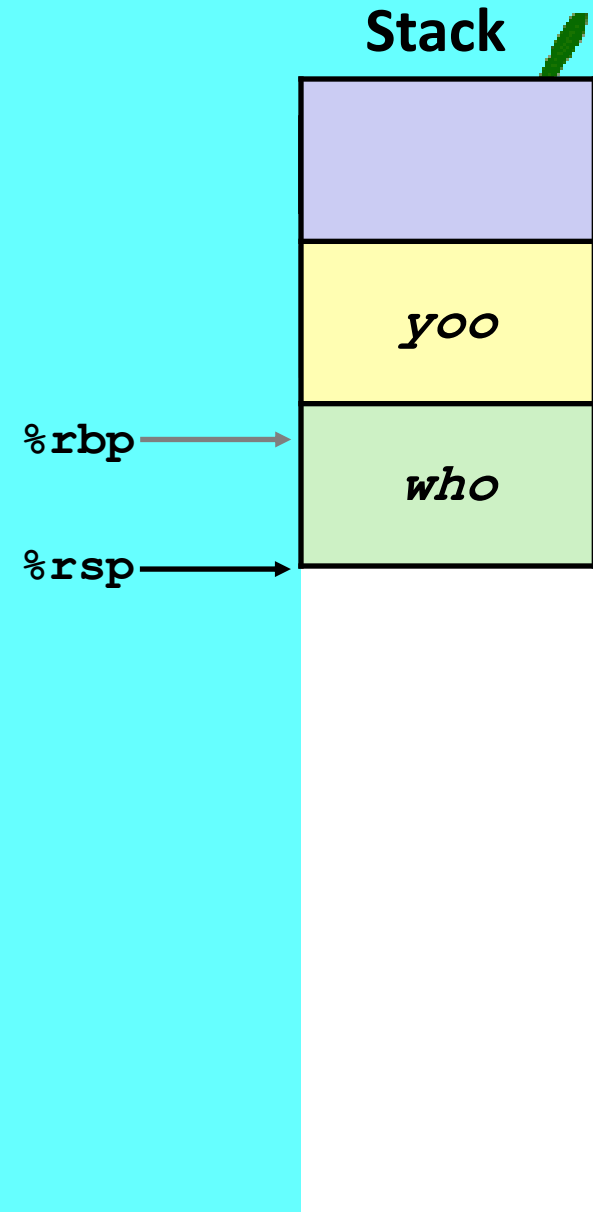
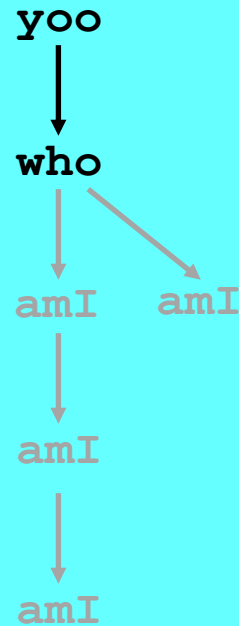
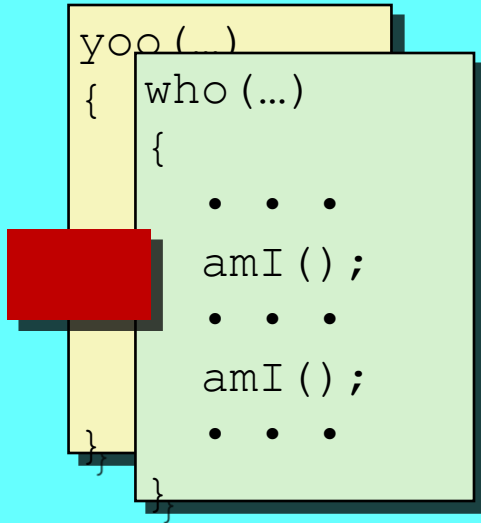
Example



Example

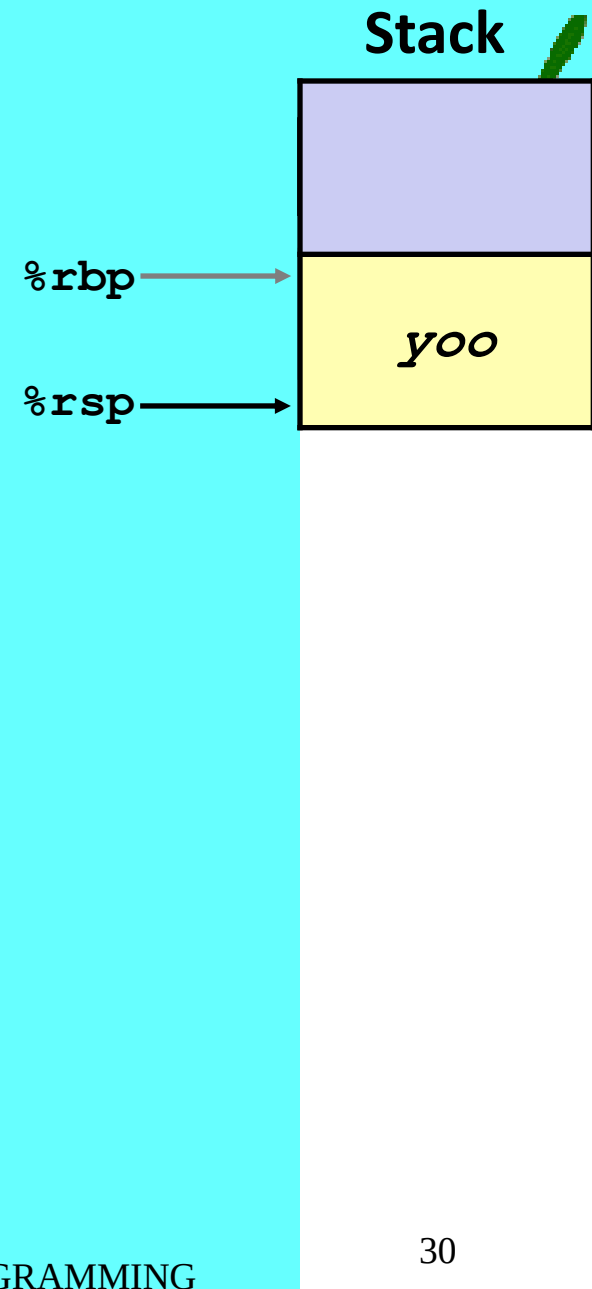
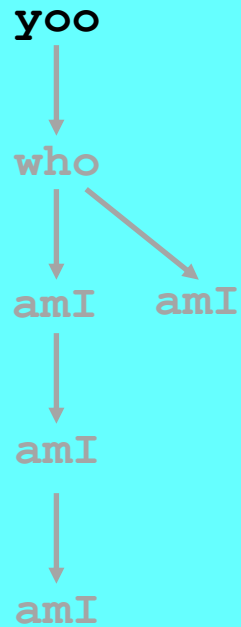


Example



Example

```
yoo (...)  
{  
  .  
  .  
  who ();  
  .  
  .  
}
```



Register Saving Conventions

- ★ When procedure **yoo** calls **who**:
 - **yoo** is the *caller*
 - **who** is the *callee*
- ★ Can register be used for temporary storage?
- ★ Conventions
 - *“Caller Saved”*
 - ★ Caller saves temporary values in its frame before the call
 - *“Callee Saved”*
 - ★ Callee saves temporary values in its frame before using them
 - ★ Callee restores them before returning to caller

x86-64 Linux Register Usage #1

* **%rax**

- Return value
- Also caller-saved
- Can be modified by procedure

* **%rdi, ..., %r9**

- Arguments
- Also caller-saved
- Can be modified by procedure

* **%r10, %r11**

- Caller-saved
- Can be modified by procedure

Return value

Arguments

Caller-saved
temporaries

%rax

%rdi

%rsi

%rdx

%rcx

%r8

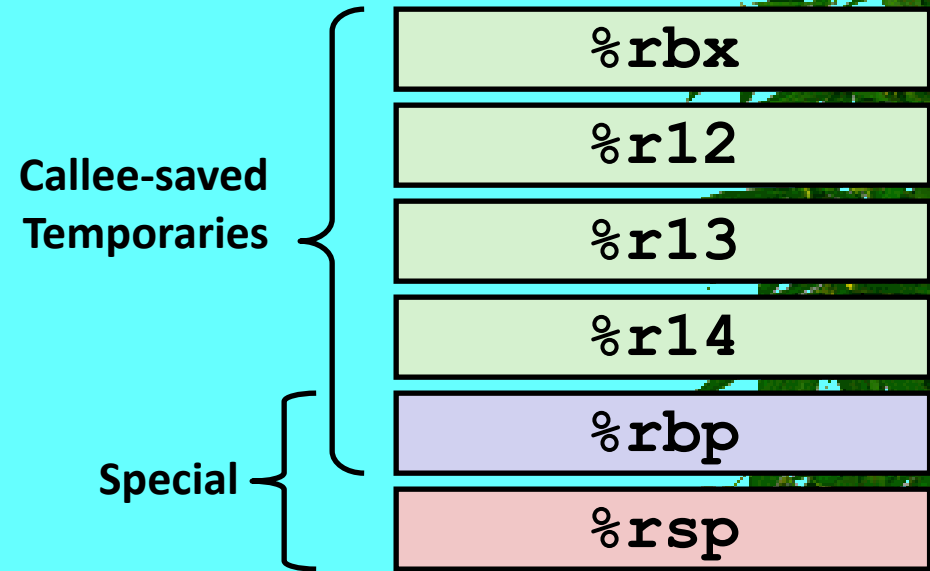
%r9

%r10

%r11

x86-64 Linux Register Usage #2

- * **%rbx, %r12, %r13, %r14**
 - Callee-saved
 - Callee must save & restore
- * **%rbp**
 - Callee-saved
 - Callee must save & restore
 - May be used as frame pointer
 - Can mix & match
- * **%rsp**
 - Special form of callee save
 - Restored to original value upon exit from procedure



SOC 2040

SYSTEM PROGRAMMING

RECURSIVE FUNCTION IMPLEMENTATION

Recursive Function

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movq    $0, %rax
    testq   %rdi, %rdi
    je      .L6
    movq    %rdi, %rbx
    andq    $1, %rbx
    pushq   %rbx
    shrq    %rdi, %rdi # (by 1)
    call    pcount_r
rets: popq   %rbx
    addq    %rbx, %rax
.L6:
    ret
```

Steps in Designing Recursive Function:

Step1 : Identify & write the termination condition for the recursive function

Step2 : Identify and write the tasks to be performed in the recursive function body

Step3 : Identify & push intermediate result onto the stack

Step4 : Identify & compute the next argument to be passed to the recursive function

Step5 : Call the recursive function with the next argument

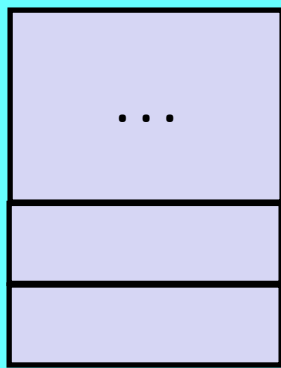
Step6 : Identify and write the tasks to be performed when the recursive function returns

Calling Recursive Function in main

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movq    $0, %rax
    testq   %rdi, %rdi
    je      .L6
    movq    %rdi, %rbx
    andq    $1, %rbx
    pushq   %rbx
    shrq    %rdi # (by 1)
    call    pcount_r
rets: popq   %rbx
      addq   %rbx, %rax
.L6:
    ret
```

Register	Use(s)	Type
<code>%rdi</code>	<code>x</code>	<i>Argument</i>
<code>%rax</code>	<i>Return value</i>	<i>Return value</i>



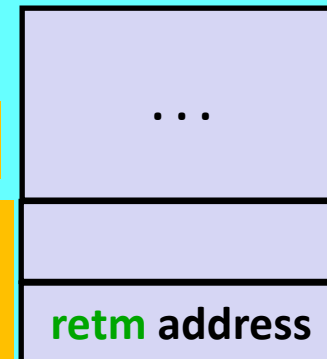
Stack Before
Recursive
function call

`x=5=00000101`

Calling main function

```
movq x, %rdi
call pcount_r
```

```
retm: movq %rax, (%rsi)
```



Stack After
Recursive
function call

Recursive Function Terminal Case

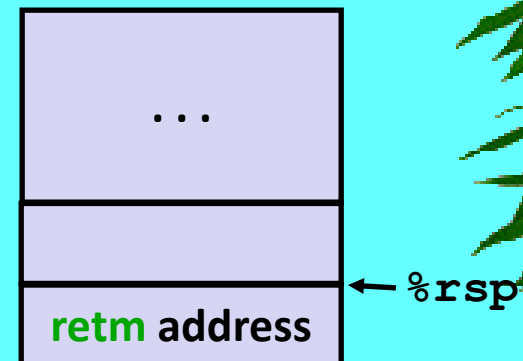
```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movq    $0, %rax
    testq   %rdi, %rdi
    je      .L6
    movq    %rdi, %rbx
    andq    $1, %rbx
    pushq   %rbx
    shrq    %rdi # (by 1)
    call    pcount_r
rets: popq   %rbx
      addq   %rbx, %rax
.L6:
    ret
```

Register	Use(s)	Type
<code>%rdi</code>	<code>x</code>	<i>Argument</i>
<code>%rax</code>	<i>Return value</i>	<i>Return value</i>

`%rdi=x=00000101`
`%rax=0`

`%rdi !=0`



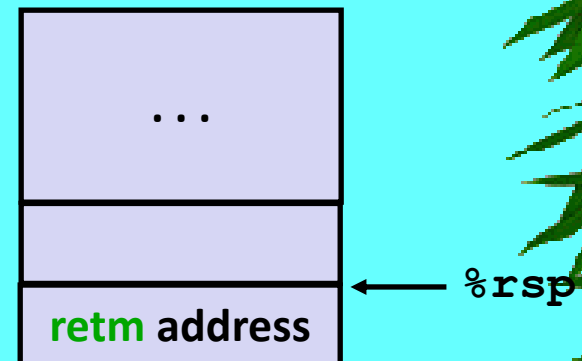
Recursive Function Register Save

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

Register	Use(s)	Type
<code>%rdi</code>	<code>x</code>	<i>Argument</i>

`%rdi = x=00000101`

```
pcount_r:
    movq    $0, %rax
    testq   %rdi, %rdi
    je      .L6
    movq    %rdi, %rbx
    andq    $1, %rbx
    pushq   %rbx
    shrq    %rdi # (by 1)
    call    pcount_r
rets: popq   %rbx
      addq   %rbx, %rax
.L6:
    ret
```



Recursive Function Call Setup

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movq    $0, %rax
    testq   %rdi, %rdi
    je      .L6
    movq    %rdi, %rbx
    andq    $1, %rbx
    pushq   %rbx
    shrq    %rdi # (by 1)
    call    pcount_r
rets: popq   %rbx
      addq   %rbx, %rax
.L6:
    ret
```

Register	Use(s)	Type
%rdi	$x \gg 1$	Rec. argument
%rbx	$x \& 1$	Callee-saved

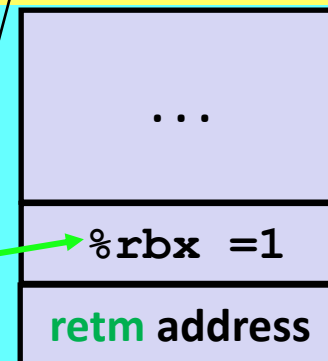
%rdi = x=00000101

%rbx=%rdi = 00000101

%rbx = %rbx & 1 = 00000101 & 00000001 = 1

pushq %rbx

%rdi = %rdi >> 1 = 00000101 >> 1 = 00000010



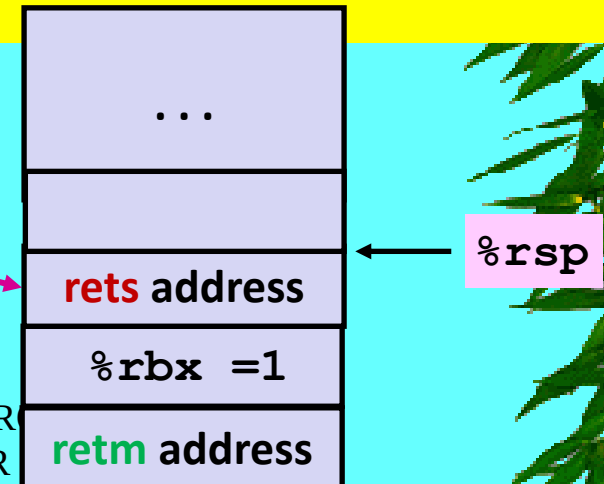
Recursive Function Call

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movq    $0, %rax
    testq   %rdi, %rdi
    je      .L6
    movq    %rdi, %rbx
    andq    $1, %rbx
    pushq   %rbx
    shrq    %rdi # (by 1)
    call    pcount_r
rets: popq   %rbx
      addq   %rbx, %rax
.L6:
    ret
```

Register	Use(s)	Type
%rbx	x & 1	Callee-saved
%rax	Recursive call return value	

call pcount_r



Recursive Function Call

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movq    $0, %rax
    testq   %rdi, %rdi
    je      .L6
    movq    %rdi, %rbx
    andq    $1, %rbx
    pushq   %rbx
    shrq    %rdi # (by 1)
    call    pcount_r
rets: popq   %rbx
    addq    %rbx, %rax
.L6:
    ret
```

Register	Use(s)	Type
<code>%rbx</code>	<code>x & 1</code>	<i>Callee-saved</i>
<code>%rax</code>	<i>Recursive call return value</i>	

call pcount_r

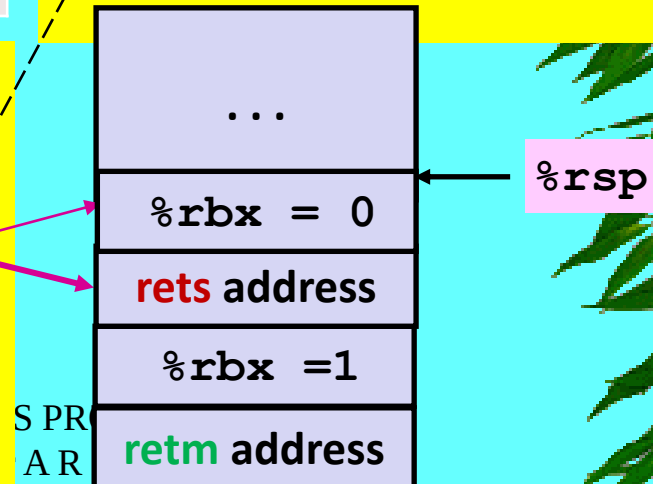
`%rdi = 00000010` `%rdi != 0`

`%rbx = %rdi = 00000010`

`%rbx = %rbx & 1 = 00000010 & 00000001 = 0`

pushq %rbx

`%rdi = %rdi >> 1 = 00000010 >> 1 = 00000001`



Recursive Function Call

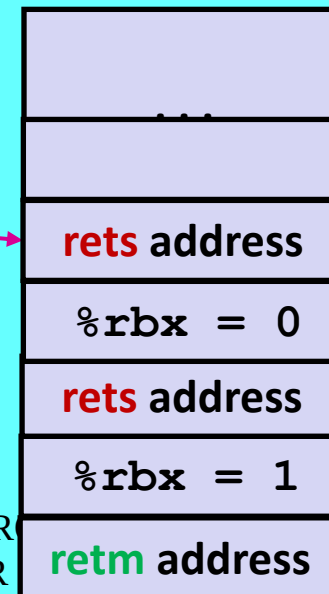
```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

pcount_r:

```
movq    $0, %rax
testq   %rdi, %rdi
je       .L6
movq     %rdi, %rbx
andq     $1, %rbx
pushq    %rbx
shrq     %rdi # (by 1)
call     pcount_r
rets: popq %rbx
      addq %rbx, %rax
.L6:
ret
```

Register	Use(s)	Type
%rbx	x & 1	Callee-saved
%rax	Recursive call return value	

call pcount_r



Recursive Function Call

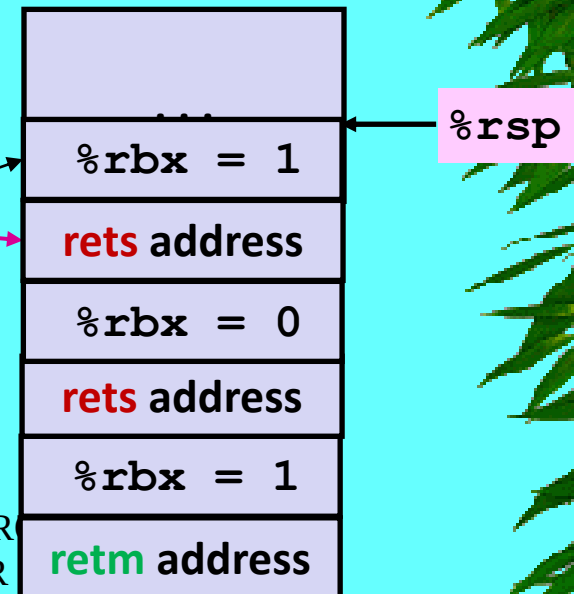
```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movq    $0, %rax
    testq   %rdi, %rdi
    je      .L6
    movq    %rdi, %rbx
    andq    $1, %rbx
    pushq   %rbx
    shrq    %rdi # (by 1)
    call    pcount_r
rets: popq   %rbx
    addq    %rbx, %rax
.L6:
    ret
```

Register	Use(s)	Type
<code>%rbx</code>	<code>x & 1</code>	<i>Callee-saved</i>
<code>%rax</code>	<i>Recursive call return value</i>	

call pcount_r
`%rdi = 00000001` `%rdi != 0`
`%rbx = %rdi = 00000001`
`%rbx = %rbx & 1 = 00000001 & 00000001 = 1`

pushq %rbx
`%rdi = %rdi >> 1 = 00000001 >> 1 = 00000000`



Recursive Function Call

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

pcount_r:

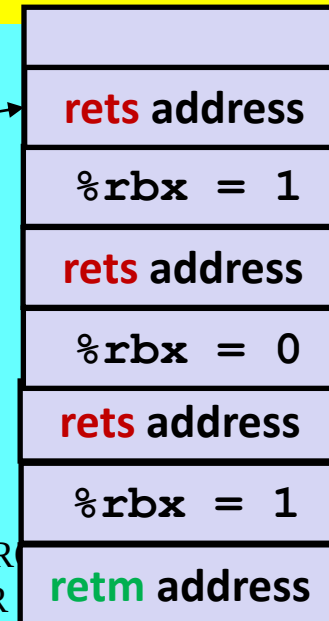
```
movq    $0, %rax
testq   %rdi, %rdi
je      .L6
movq    %rdi, %rbx
andq    $1, %rbx
pushq   %rbx
shrq    %rdi # (by 1)
call    pcount_r
```

```
rets: popq %rbx
      addq %rbx, %rax
```

.L6: ret

Register	Use(s)	Type
%rbx	$x \& 1$	Callee-saved
%rax	Recursive call return value	

call pcount_r



Recursive Function Call

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movq    $0, %rax
    testq   %rdi, %rdi
    je      .L6
    movq    %rdi, %rbx
    andq    $1, %rbx
    pushq   %rbx
    shrq    %rdi # (by 1)
    call    pcount_r
rets: popq   %rbx
      addq   %rbx, %rax
.L6:  ret
```

Register	Use(s)	Type
%rbx	x & 1	Callee-saved
%rax	Recursive call return value	

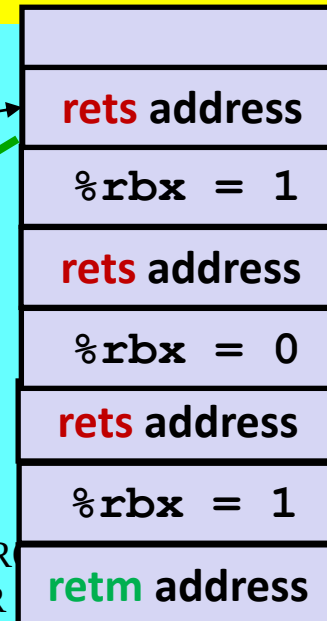
call pcount_r

%rdi = 00000000 %rbx = 1 %rdi == 0

jump to L6

ret

%rip



%rsp

Recursive Function Call

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

pcount_r:

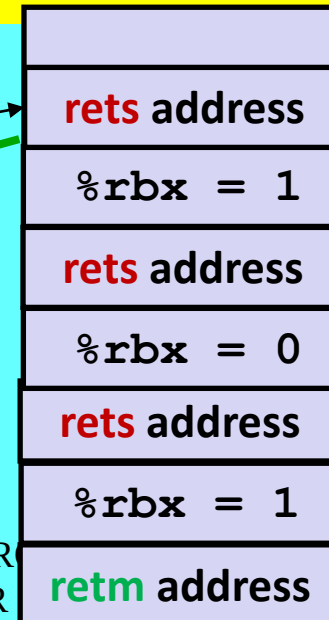
```
movq    $0, %rax
testq   %rdi, %rdi
je      .L6
movq    %rdi, %rbx
andq    $1, %rbx
pushq   %rbx
shrq    %rdi # (by 1)
call    pcount_r
```

```
rets: popq %rbx
      addq %rbx, %rax
```

.L6: ret

Register	Use(s)	Type
%rbx	x & 1	Callee-saved
%rax	Recursive call return value	

ret ← %rip= rets address



Recursive Function Return

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movq    $0, %rax
    testq   %rdi, %rdi
    je      .L6
    movq    %rdi, %rbx
    andq    $1, %rbx
    pushq   %rbx
    shrq    %rdi # (by 1)
    call    pcount_r
rets: popq   %rbx
      addq   %rbx, %rax
.L6:  ret
```

Register	Use(s)	Type
<code>%rbx</code>	<code>x & 1</code>	<i>Callee-saved</i>
<code>%rax</code>	<i>Recursive call return value</i>	

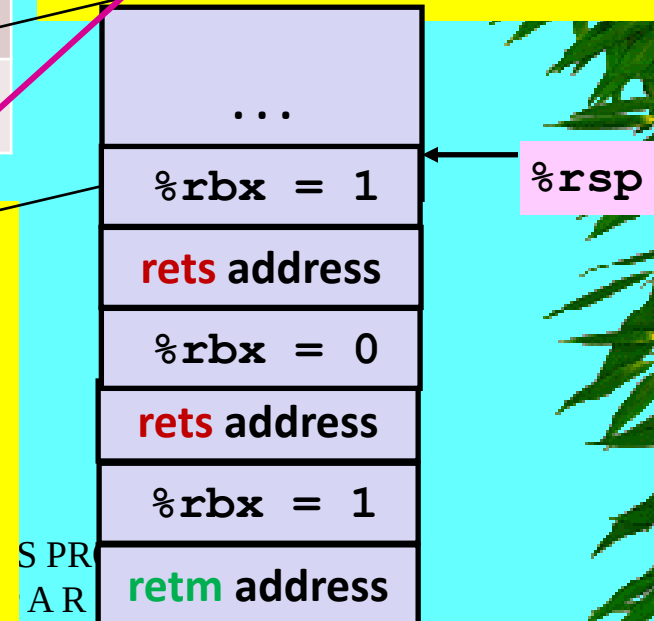
ret

`popq %rbx`
`%rbx=1`

`%rax=%rax + %rbx = 0 + 1 = 1`

`%rip= rets address`

`%rbx`



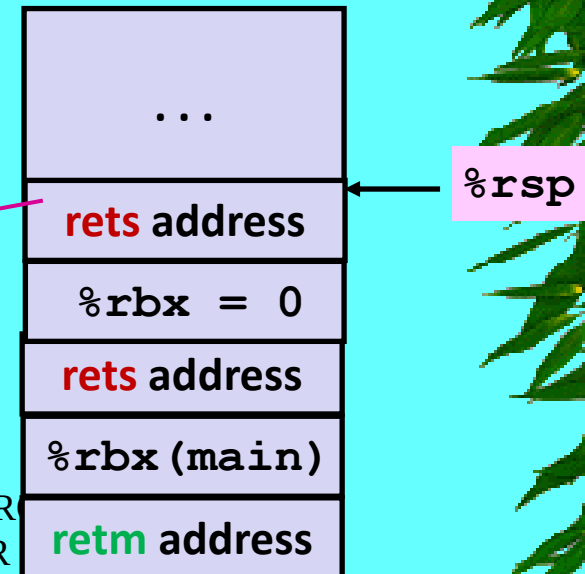
Recursive Function Return

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movq    $0, %rax
    testq   %rdi, %rdi
    je      .L6
    movq    %rdi, %rbx
    andq    $1, %rbx
    pushq   %rbx
    shrq    %rdi # (by 1)
    call    pcount_r
rets: popq   %rbx
      addq   %rbx, %rax
.L6: ret
```

Register	Use(s)	Type
<code>%rbx</code>	<code>x & 1</code>	<i>Callee-saved</i>
<code>%rax</code>	<i>Recursive call return value</i>	

ret %rip =rets address



Recursive Function Return

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

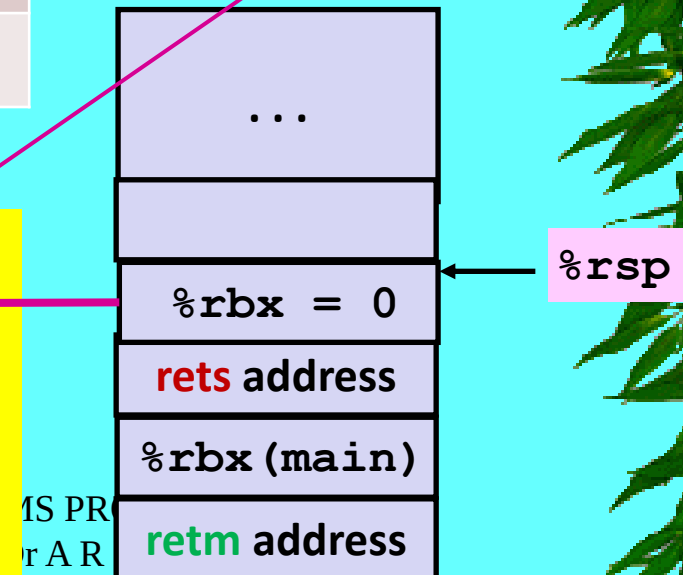
```
pcount_r:
    movq    $0, %rax
    testq   %rdi, %rdi
    je      .L6
    movq    %rdi, %rbx
    andq    $1, %rbx
    pushq   %rbx
    shrq    %rdi # (by 1)
    call    pcount_r
rets: popq   %rbx
      addq   %rbx, %rax
.L6:  ret
```

Register	Use(s)	Type
<code>%rbx</code>	<code>x & 1</code>	<i>Callee-saved</i>
<code>%rax</code>	<i>Recursive call return value</i>	

`popq %rbx`

`%rbx=0`

`%rax=%rax + %rbx = 1 + 0 = 1`



MS PR
r AR

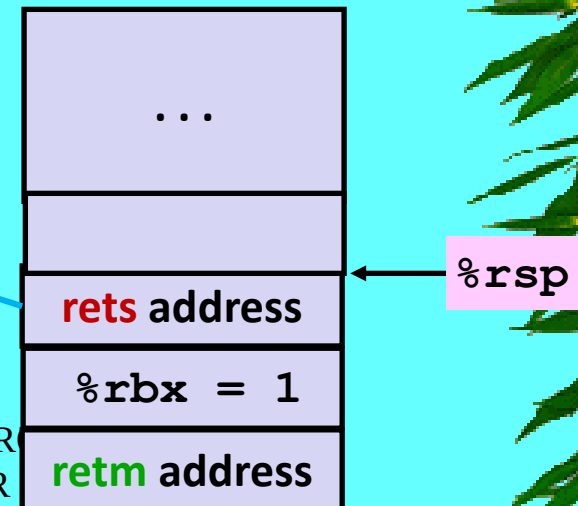
Recursive Function Return

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movq    $0, %rax
    testq   %rdi, %rdi
    je      .L6
    movq    %rdi, %rbx
    andq    $1, %rbx
    pushq   %rbx
    shrq    %rdi # (by 1)
    call    pcount_r
    rets:   popq    %rbx
          addq    %rbx, %rax
.L6:      ret
```

Register	Use(s)	Type
<code>%rbx</code>	<code>x & 1</code>	<i>Callee-saved</i>
<code>%rax</code>	<i>Recursive call return value</i>	

`ret` %rip =rets address



Recursive Function Return

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

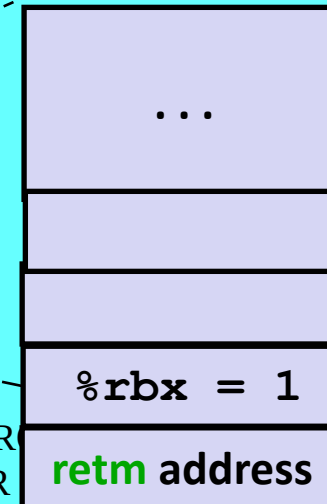
```
pcount_r:
    movq    $0, %rax
    testq   %rdi, %rdi
    je      .L6
    movq    %rdi, %rbx
    andq    $1, %rbx
    pushq   %rbx
    shrq    %rdi # (by 1)
    call    pcount_r
rets: popq   %rbx
      addq   %rbx, %rax
.L6:  ret
```

Register	Use(s)	Type
<code>%rbx</code>	<code>x & 1</code>	<i>Callee-saved</i>
<code>%rax</code>	<i>Recursive call return value</i>	

`popq %rbx`

`%rax = %rax + %rbx = 1 + 1 = 2`

`%rbx = 1`



Recursive Function Return

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movq    $0, %rax
    testq   %rdi, %rdi
    je      .L6
    movq    %rdi, %rbx
    andq    $1, %rbx
    pushq   %rbx
    shrq    %rdi # (by 1)
    call    pcount_r
rets: popq   %rbx
      addq   %rbx, %rax
.L6:  ret
```

Register	Use(s)	Type
<code>%rbx</code>	<code>x & 1</code>	<i>Callee-saved</i>
<code>%rax</code>	<i>Recursive call return value</i>	

ret

%rip = retm address



%rsp

Recursive Function Return

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

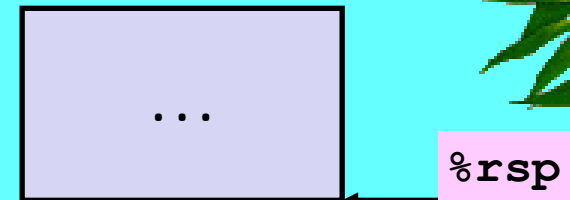
```
pcount_r:
    movq    $0, %rax
    testq   %rdi, %rdi
    je      .L6
    pushq   %rbx
    movq    %rdi, %rbx
    andq    $1, %rbx
    shrq    %rdi # (by 1)
    call    pcount_r
rets: addq   %rbx, %rax
      popq   %rbx
.L6:      ret
```

Register	Use(s)	Type
<code>%rbx</code>	<code>x & 1</code>	<i>Callee-saved</i>
<code>%rax</code>	<i>Recursive call return value</i>	

ret **%rip = retm address**
return to main function

Calling main function

`movq x, %rdi`
`call pcount_r`
retm: `movq %rax, (%rsi)`




```
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
<code>%rax</code>	<i>n</i>	<i>Argument</i>
<code>%rax</code>	<i>Return value</i>	<i>Return value</i>

```
%rax = %rbx = n = 8
```

28 March 2024

Label	Instruction	Comment
.global	_start	
	.data	
n:	.quad 0x00000000000000008	
nfact:	.quad 0x00000000000000000	
	.text	
_start:	movq n, %rbx	
	movq %rbx,%rax	
	call ffact	
retm:	movq \$nfact, %rdi	
	movq %rax, (%rdi)	
	movq \$60, %rax	
	xorq %rdi, %rdi	
	syscall	
ffact:	cmpq \$1, %rbx	
	je L1	
	decq %rbx	
	pushq %rbx	
	call ffact	
rets:	popq %rbx	
	mulq %rbx	
L1:	ret	

```
%rax= 8 * 1 * 2 * 3 * 4 * 5 * 6 * 7
      = 40320
```

%rsp

rets address

%rbx=1 (n-7)

rets address

%rbx=2 (n-6)

rets address

%rbx=3 (n-5)

rets address

%rbx=4 (n-4)

rets address

%rbx=5 (n-3)

rets address

%rbx=6 (n-2)

rets address

~~%rbx~~ = 7 (n-1)

retm address

Recursive Function

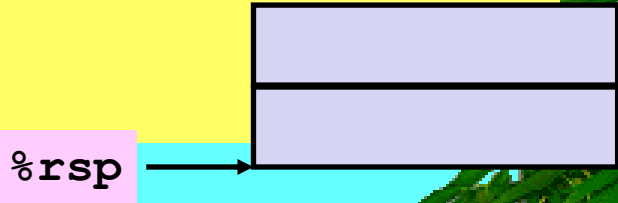
```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

%rax = %rbx = n = 8

%rsp points to Top of the Stack

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq  n, %rbx
    movq  %rbx,%rax
    call  ffact
retm:   movq  $nfact, %rdi
        movq  %rax, (%rdi)
        movq  $60, %rax
        xorq  %rdi, %rdi
        syscall
ffact:  cmpq  $1, %rbx
        je    L1
        decq  %rbx
        pushq %rbx
        call  ffact
rets:   popq  %rbx
        mulq  %rbx
L1:     ret
```



Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
<code>%rax</code>	<code>n</code>	<i>Argument</i>
<code>%rax</code>	<i>Return value</i>	<i>Return value</i>

`%rax = %rbx = n = 8`

Return address `retm` pushed onto the stack

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:    movq    $nfact, %rdi
    movq    %rax, (%rdi)
    movq    $60, %rax
    xorq    %rdi, %rdi
    syscall
ffact:   cmpq    $1, %rbx
    je      L1
    decq    %rbx
    pushq   %rbx
    call    ffact
rets:    popq    %rbx
    mulq    %rbx
L1:      ret
```

`%rsp`

`retm` address

Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

%rax = %rbx = n = 8

%rbx = 8 - 1 = 7
pushq %rbx

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
          movq    %rax, (%rdi)
          movq    $60, %rax
          xorq    %rdi, %rdi
          syscall
ffact:    cmpq    $1, %rbx
          je      L1
          decq    %rbx
          pushq   %rbx
          call    ffact
rets:     popq    %rbx
          mulq    %rbx
L1:        ret
```

%rsp →

%rbx=7 (n-1)

retm address

Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

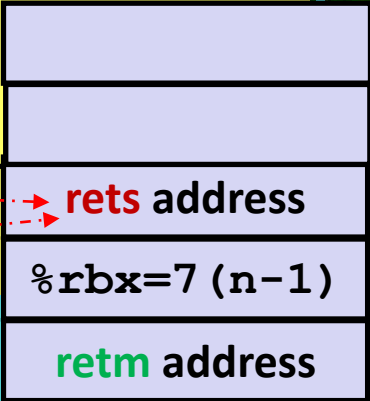
Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

%rax = n = 8
%rbx = n -1 = 7

call ffact
Return address rets pushed onto stack

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
    movq    %rax, (%rdi)
    movq    $60, %rax
    xorq    %rdi, %rdi
    syscall
ffact:    cmpq    $1, %rbx
    je      L1
    decq    %rbx
    pushq   %rbx
    call    ffact
rets:     popq    %rbx
    mulq    %rbx
L1:        ret
```

%rsp



Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

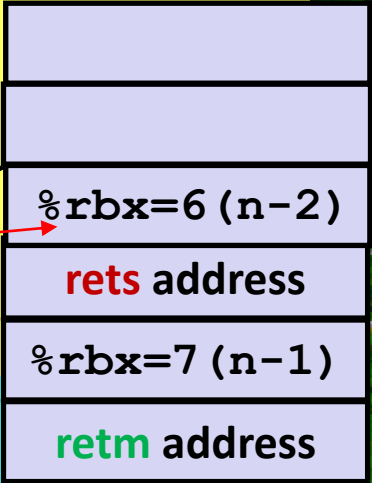
Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
          movq    %rax, (%rdi)
          movq    $60, %rax
          xorq    %rdi, %rdi
          syscall
ffact:    cmpq    $1, %rbx
          je      L1
          decq    %rbx
          pushq   %rbx
          call    ffact
          popq    %rbx
          mulq    %rbx
L1:       ret
```

%rax = n = 8
%rbx = n -1 = 7

%rbx = 7 -1 = 6
pushq %rbx

%rsp



Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

%rax = n = 8
%rbx = n -2 = 6

call ffact
Return address **rets** pushed onto stack

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
          movq    %rax, (%rdi)
          movq    $60, %rax
          xorq    %rdi, %rdi
          syscall
ffact:    cmpq    $1, %rbx
          je      L1
          decq    %rbx
          pushq   %rbx
          call    ffact
rets:     popq    %rbx
          mulq    %rbx
L1:       ret
```

%rsp →

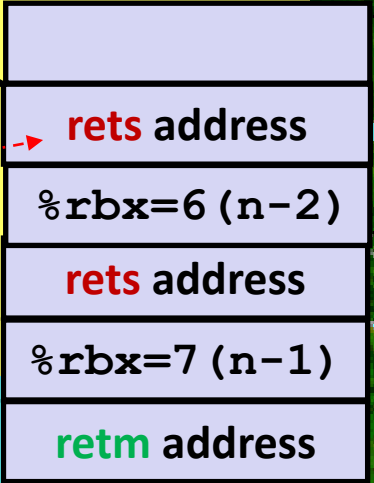
rets address

%rbx=6 (n-2)

rets address

%rbx=7 (n-1)

retm address



Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

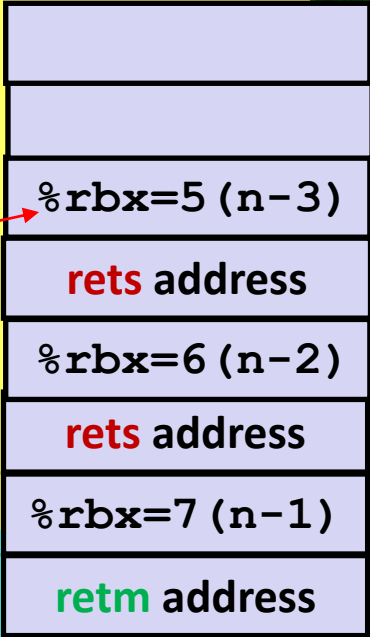
Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

%rax = n = 8
%rbx = n -2 = 6

%rbx = 6-1 = 5
pushq %rbx

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
          movq    %rax, (%rdi)
          movq    $60, %rax
          xorq    %rdi, %rdi
          syscall
ffact:    cmpq    $1, %rbx
          je      L1
          decq    %rbx
          pushq   %rbx
          call    ffact
rets:     popq    %rbx
          mulq    %rbx
L1:       ret
```

%rsp →



Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

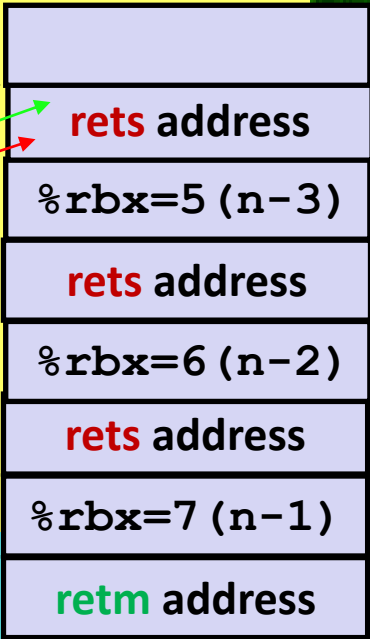
Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

%rax = n = 8
%rbx = n -3 = 5

call ffact
Return address **rets** pushed onto stack

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
          movq    %rax, (%rdi)
          movq    $60, %rax
          xorq    %rdi, %rdi
          syscall
ffact:    cmpq    $1, %rbx
          je      L1
          decq    %rbx
          pushq   %rbx
          call    ffact
          popq    %rbx
          mulq    %rbx
L1:       ret
```

%rsp



Recursive Function

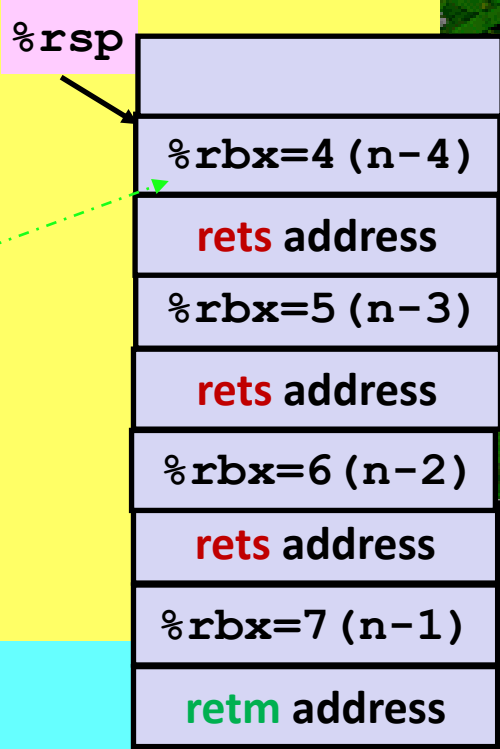
```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

%rax = n = 8
%rbx = n -3 = 5

%rbx = 5-1 = 4
pushq %rbx

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
          movq    %rax, (%rdi)
          movq    $60, %rax
          xorq    %rdi, %rdi
          syscall
ffact:    cmpq    $1, %rbx
          je      L1
          decq    %rbx
          pushq   %rbx
          call    ffact
rets:     popq    %rbx
          mulq    %rbx
L1:       ret
```

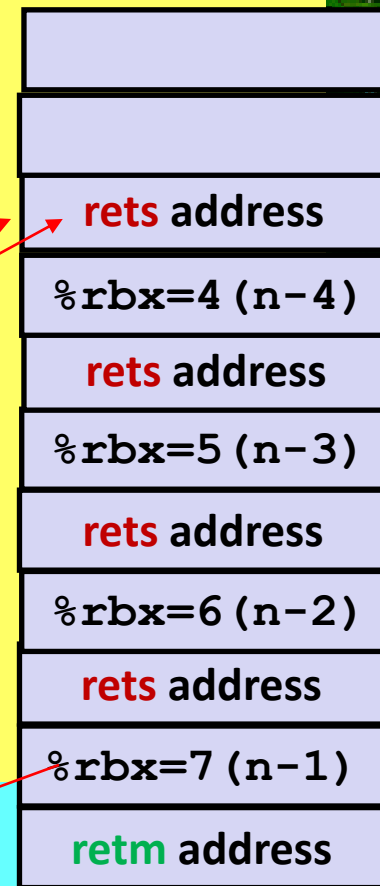



```
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

```
%rax = n = 8
%rbx = n - 4 = 4
```

call ffact
Return address **rets** pushed onto
stack

%rsp



Recursive Function

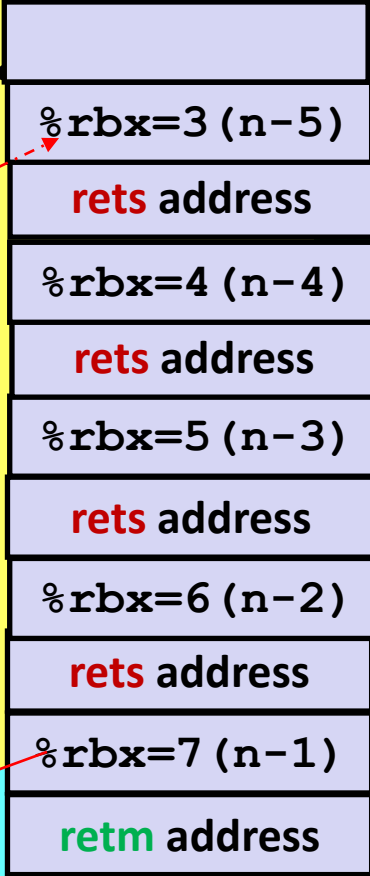
```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

%rax = n = 8
%rbx = n - 4 = 4

%rbx = 4 - 1 = 3
pushq %rbx

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
          movq    %rax, (%rdi)
          movq    $60, %rax
          xorq    %rdi, %rdi
          syscall
ffact:    cmpq    $1, %rbx
          je      L1
          decq    %rbx
          pushq   %rbx
          call    ffact
rets:     popq    %rbx
          mulq    %rbx
L1:        ret
```



Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

%rax = n = 8
%rbx = n - 5 = 3

call ffact
Return address **rets** pushed onto stack

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:      movq    $nfact, %rdi
            movq    %rax, (%rdi)
            movq    $60, %rax
            xorq    %rdi, %rdi
            syscall
ffact:     cmpq    $1, %rbx
            je      L1
            decq    %rbx
            pushq   %rbx
            call    ffact
            popq    %rbx
            mulq    %rbx
L1:         ret
```

%rsp

rets address

%rbx=3 (n-5)

rets address

%rbx=4 (n-4)

rets address

%rbx=5 (n-3)

rets address

%rbx=6 (n-2)

rets address

%rbx=7 (n-1)

retm address

Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

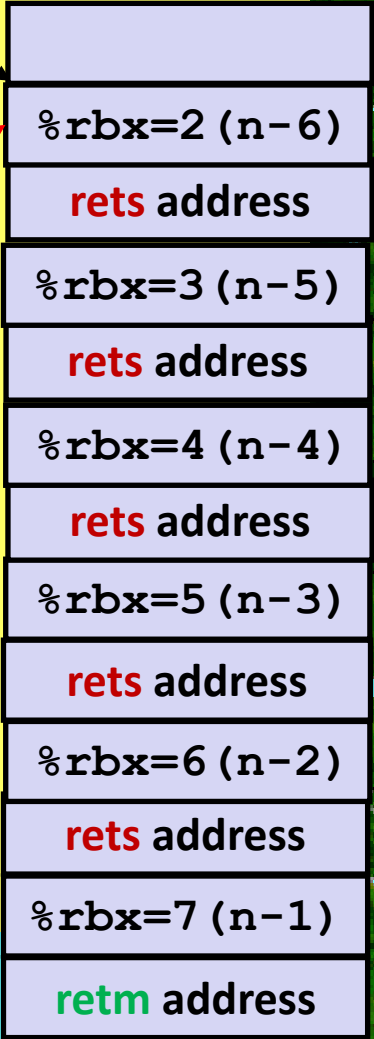
Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

%rax = n = 8
%rbx = n - 5 = 3

%rbx = 3 - 1 = 2
pushq %rbx

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
          movq    %rax, (%rdi)
          movq    $60, %rax
          xorq    %rdi, %rdi
          syscall
ffact:    cmpq    $1, %rbx
          je      L1
          decq    %rbx
          pushq   %rbx
          call    ffact
rets:     popq    %rbx
          mulq    %rbx
L1:       ret
```

%rsp



Recursive Function

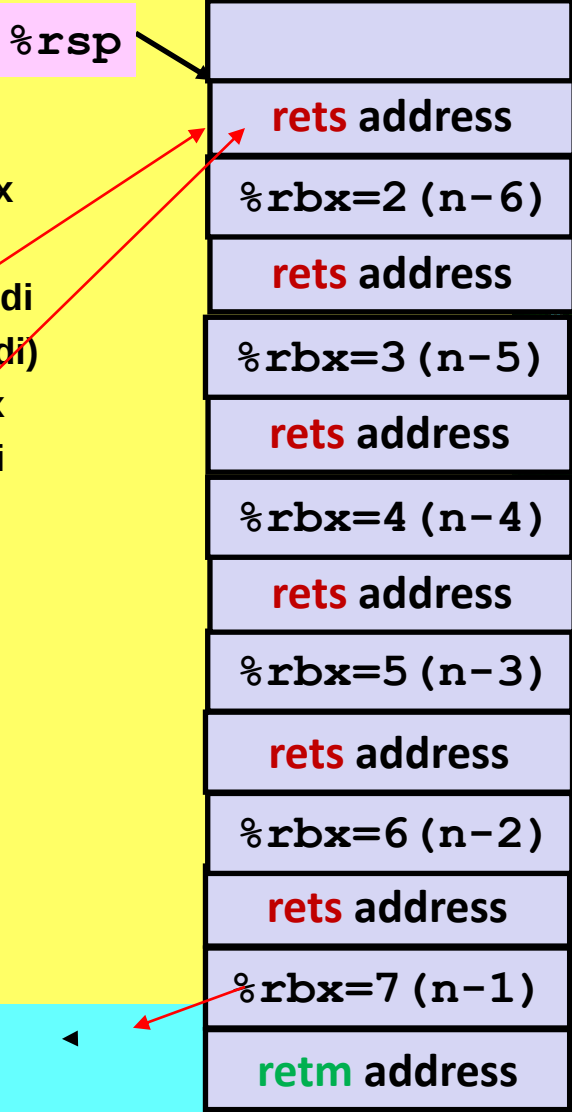
```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

%rax = n = 8
%rbx = n - 6 = 2

call ffact
Return address **rets** pushed onto stack

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
    movq    %rax, (%rdi)
    movq    $60, %rax
    xorq    %rdi, %rdi
    syscall
ffact:    cmpq    $1, %rbx
    je      L1
    decq    %rbx
    pushq   %rbx
    call    ffact
    popq    %rbx
    mulq    %rbx
L1:       ret
```



Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

%rax = n = 8
%rbx = n - 6 = 2

%rbx = 2 - 1 = 1
pushq %rbx

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
          movq    %rax, (%rdi)
          movq    $60, %rax
          xorq    %rdi, %rdi
          syscall
ffact:    cmpq    $1, %rbx
          je      L1
          decq    %rbx
          pushq   %rbx
          call    ffact
rets:     popq    %rbx
          mulq    %rbx
L1:       ret
```

%rsp

%rbx=1 (n-7)

rets address

%rbx=2 (n-6)

rets address

%rbx=3 (n-5)

rets address

%rbx=4 (n-4)

rets address

%rbx=5 (n-3)

rets address

%rbx=6 (n-2)

rets address

%rbx=7 (n-1)

retm address

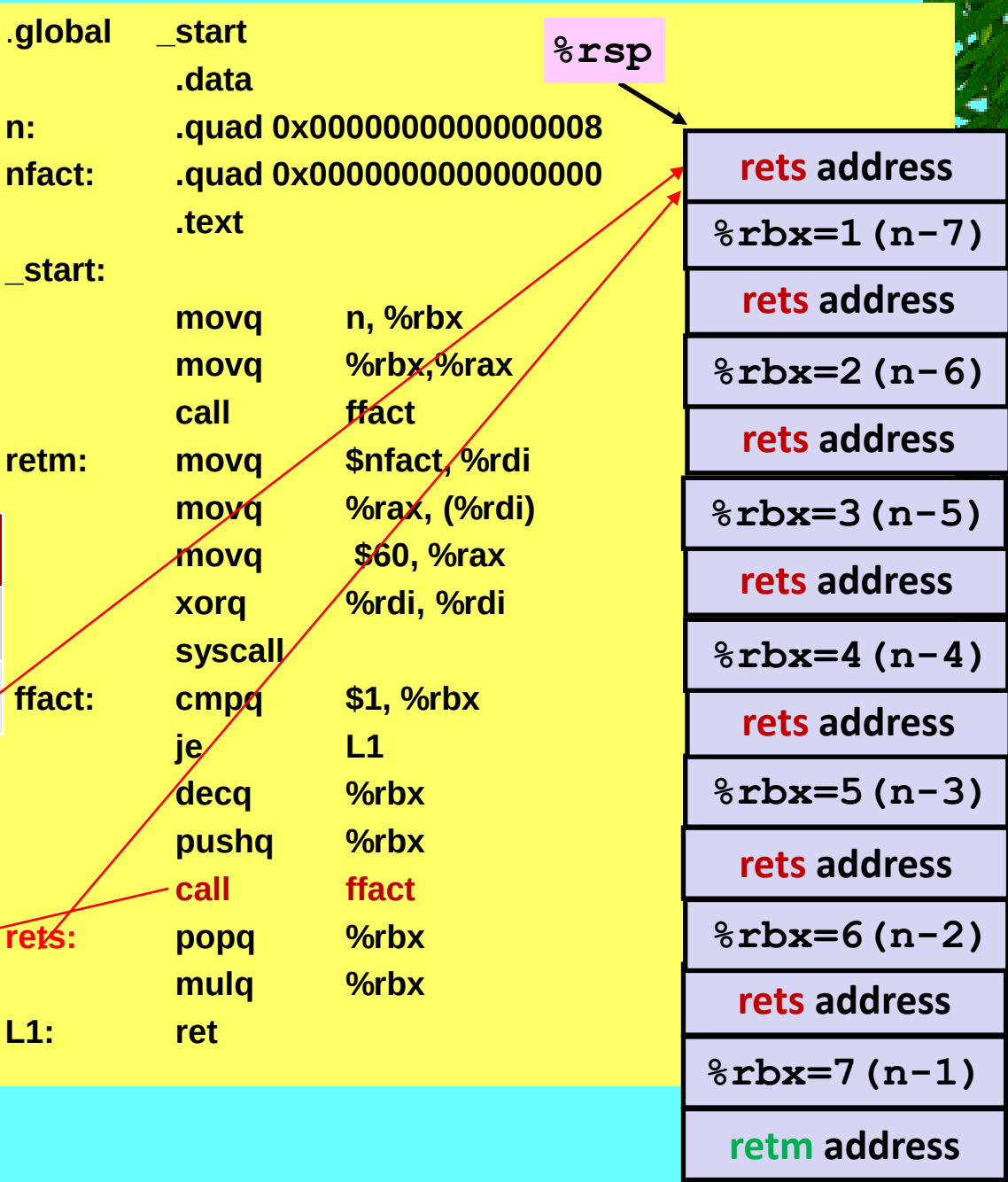
Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

%rax = n = 8
%rbx = n - 7 = 1

call ffact
Return address **rets** pushed onto stack




```
/* Recursive function ffact */  
int ffact( int n)  
{  
    if (n == 1)  
        return 1;  
    else  
        return (n*ffact(n-1));  
}
```

Register	Use(s)	Type
<code>%rax</code>	<i>n</i>	<i>Argument</i>
<code>%rax</code>	<i>Return value</i>	<i>Return value</i>

`%rax = n = 8`

```
%rbx = 1
ret to rets
popq %rbx    %rbx = 1
Mulq %rax → %rax = %rax * %rbx
```

.global	_start				
	.data				
n:	.quad	0x000000000000000008			
nfact:	.quad	0x000000000000000000			
	.text				
_start:					
	movq	n, %rbx			
	movq	%rbx,%rax			
	call	ffact			
retm:	movq	\$nfact,%rdi			
	movq	%rax, (%rdi)			
	movq	\$60, %rax			
	xorq	%rdi, %rdi			
	syscall				
ffact:	cmpq	\$1, %rbx			
	je	L1			
	decq	%rbx			
	pushq	%rbx			
	call	ffact			
rets:	popq	%rbx			
	mulq	%rbx			
L1:	ret				

%rsp

%rbx=1 (n-7)
rets address

%rbx=2 (n-6)
rets address

%rbx=3 (n-5)
rets address

%rbx=4 (n-4)
rets address

%rbx=5 (n-3)
rets address

%rbx=6 (n-2)
rets address

%rbx=7 (n-1)
rets address

```
%rax = 8*1
```

%rsp

%rbx=1 (n-7)

rets address
%rbx=2 (n-6)

rets address
%rbx=3 (n-5)

rets address
%rbx=4 (n-4)

rets address
%rbx=5 (n-3)

rets address
%rbx=6 (n-2)

rets address
%rbx=7 (n-1)

retm address

Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

%rax = n = 8

ret to rets

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
          movq    %rax, (%rdi)
          movq    $60, %rax
          xorq    %rdi, %rdi
          syscall
ffact:    cmpq    $1, %rbx
          je      L1
          decq    %rbx
          pushq   %rbx
          call    ffact
          popq    %rbx
          mulq    %rbx
L1:       ret
```

%rsp

rets address

%rbx=2 (n-6)

rets address

%rbx=3 (n-5)

rets address

%rbx=4 (n-4)

rets address

%rbx=5 (n-3)

rets address

%rbx=6 (n-2)

rets address

%rbx=7 (n-1)

retm address

%rax = 8*1

Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
          movq    %rax, (%rdi)
          movq    $60, %rax
          xorq    %rdi, %rdi
          syscall
ffact:    cmpq    $1, %rbx
          je      L1
          decq    %rbx
          pushq   %rbx
          call    ffact
rets:     popq    %rbx
          mulq    %rbx
L1:       ret
```

%rsp

popq %rbx %rbx = 2
mulq %rbx → %rax = %rax * %rbx

%rax = 8*1*2

Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
          movq    %rax, (%rdi)
          movq    $60, %rax
          xorq    %rdi, %rdi
          syscall
ffact:    cmpq    $1, %rbx
          je      L1
          decq    %rbx
          pushq   %rbx
          call    ffact
          popq    %rbx
          mulq    %rbx
rets:     ret
L1:       ret
```

%rsp

rets address

%rbx=3 (n-5)

rets address

%rbx=4 (n-4)

rets address

%rbx=5 (n-3)

rets address

%rbx=6 (n-2)

rets address

%rbx=7 (n-1)

retm address

ret to rets

%rax = 8*1*2

Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
          movq    %rax, (%rdi)
          movq    $60, %rax
          xorq    %rdi, %rdi
          syscall
ffact:    cmpq    $1, %rbx
          je      L1
          decq    %rbx
          pushq   %rbx
          call    ffact
          popq    %rbx
          mulq    %rbx
L1:       ret
```

%rsp points to the top of the stack frame.

Stack Frame:

- %rbx=3 (n-5)
- rets address
- %rbx=4 (n-4)
- rets address
- %rbx=5 (n-3)
- rets address
- %rbx=6 (n-2)
- rets address
- %rbx=7 (n-1)
- retm address

Execution Flow:

- rets:** popq %rbx → %rbx = 3
- L1:** mulq %rbx → %rax = %rax * %rbx

popq %rbx %rbx = 3
mulq %rbx → %rax = %rax * %rbx

%rax = 8*1*2*3

Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
          movq    %rax, (%rdi)
          movq    $60, %rax
          xorq    %rdi, %rdi
          syscall
ffact:    cmpq    $1, %rbx
          je      L1
          decq    %rbx
          pushq   %rbx
          call    ffact
          popq    %rbx
          mulq    %rbx
L1:       ret
```

%rsp

rets address

%rbx=4 (n-4)

rets address

%rbx=5 (n-3)

rets address

%rbx=6 (n-2)

rets address

%rbx=7 (n-1)

retm address

ret to rets

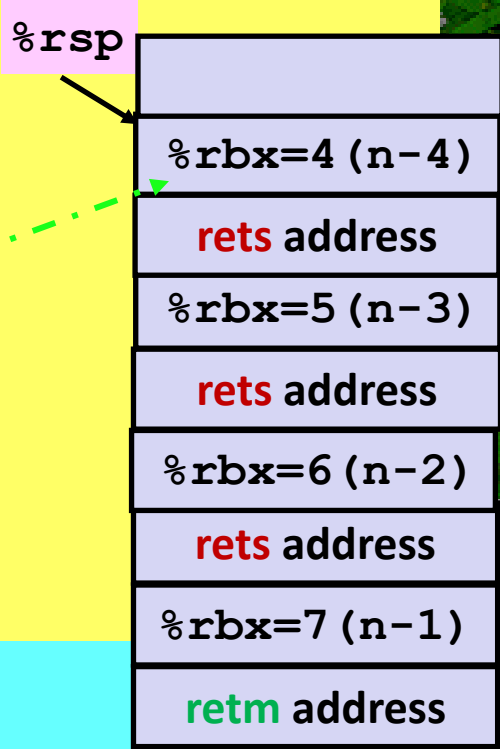
%rax = 8*1*2*3

Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:    movq    $nfact, %rdi
        movq    %rax, (%rdi)
        movq    $60, %rax
        xorq    %rdi, %rdi
        syscall
ffact:   cmpq    $1, %rbx
        je      L1
        decq    %rbx
        pushq   %rbx
        call    ffact
rets:    popq    %rbx
        mulq    %rbx
L1:      ret
```



popq %rbx %rbx = 4
mulq %rbx → %rax = %rax * %rbx

%rax = 8*1*2*3*4

Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
          movq    %rax, (%rdi)
          movq    $60, %rax
          xorq    %rdi, %rdi
          syscall
ffact:    cmpq    $1, %rbx
          je      L1
          decq    %rbx
          pushq   %rbx
          call    ffact
          popq    %rbx
          mulq    %rbx
L1:       ret
```

%rsp points to the stack frame containing:

- rets address
- %rbx=5 (n-3)
- rets address
- %rbx=6 (n-2)
- rets address
- %rbx=7 (n-1)
- retm address

rets: points to the first **rets address** entry.

L1: points to the **ret** instruction.

ret to rets

$\%rax = 8 \cdot 1 \cdot 2 \cdot 3 \cdot 4$

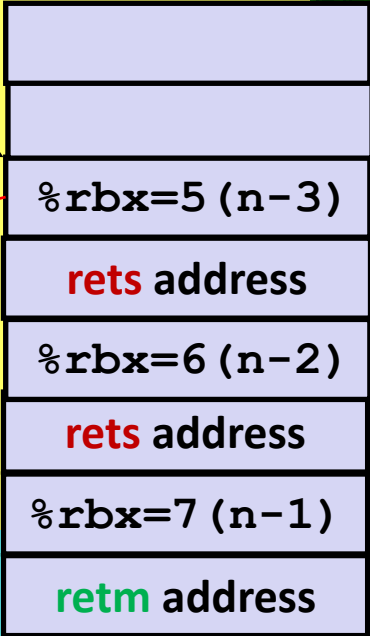
Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
          movq    %rax, (%rdi)
          movq    $60, %rax
          xorq    %rdi, %rdi
          syscall
ffact:    cmpq    $1, %rbx
          je      L1
          decq    %rbx
          pushq   %rbx
          call    ffact
rets:     popq    %rbx
          mulq    %rbx
L1:       ret
```

%rsp



popq %rbx %rbx = 5
mulq %rbx → %rax = %rax * %rbx

%rax = 8*1*2*3*4*5

Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
          movq    %rax, (%rdi)
          movq    $60, %rax
          xorq    %rdi, %rdi
          syscall
ffact:    cmpq    $1, %rbx
          je      L1
          decq    %rbx
          pushq   %rbx
          call    ffact
          popq    %rbx
          mulq    %rbx
L1:       ret
```

%rsp →

rets address

%rbx=6 (n-2)

rets address

%rbx=7 (n-1)

retm address

rets: →

L1: →

ret

ret to rets

%rax = 8*1*2*3*4*5

Recursive Function

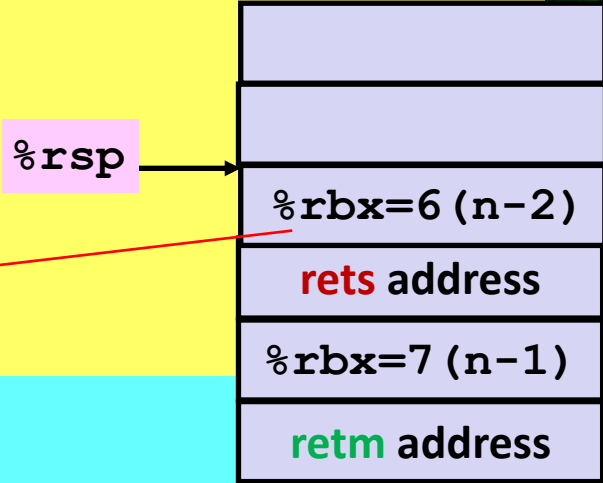
```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
          movq    %rax, (%rdi)
          movq    $60, %rax
          xorq    %rdi, %rdi
          syscall
ffact:    cmpq    $1, %rbx
          je      L1
          decq    %rbx
          pushq   %rbx
          call    ffact
          popq    %rbx
          mulq    %rbx
L1:       ret
```

popq %rbx %rbx = 6
mulq %rbx → %rax = %rax * %rbx

%rax = 8*1*2*3*4*5*6

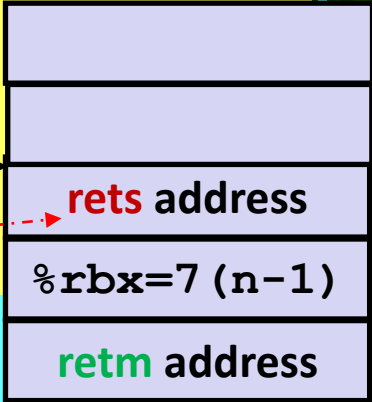


Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:     movq    $nfact, %rdi
    movq    %rax, (%rdi)
    movq    $60, %rax
    xorq    %rdi, %rdi
    syscall
ffact:    cmpq    $1, %rbx
    je      L1
    decq    %rbx
    pushq   %rbx
    call    ffact
    popq    %rbx
    mulq    %rbx
rets:
L1:       -ret
```



28 March 2024

ret to rets

$\%rax = 8 \cdot 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 \cdot 6$

Recursive Function

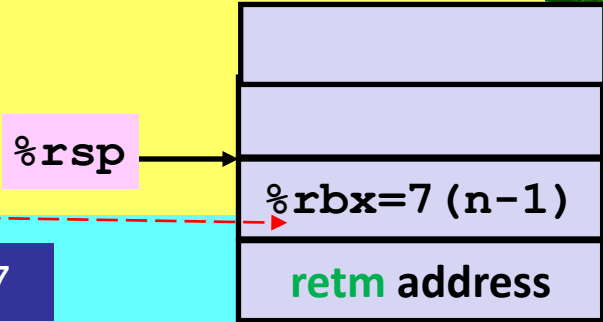
```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:    movq    $nfact, %rdi
        movq    %rax, (%rdi)
        movq    $60, %rax
        xorq    %rdi, %rdi
        syscall
ffact:   cmpq    $1, %rbx
        je      L1
        decq    %rbx
        pushq   %rbx
        call    ffact
rets:    popq     %rbx
        mulq     %rbx
L1:      ret
```

popq %rbx %rbx = 7
mulq %rbx → %rax = %rax * %rbx

%rax = 8*1*2*3*4*5*6*7



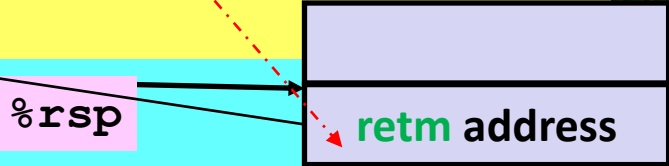
Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
    movq    $nfact, %rdi
    movq    %rax, (%rdi)
    movq    $60, %rax
    xorq    %rdi, %rdi
    syscall
ffact:   cmpq    $1, %rbx
        je      L1
        decq    %rbx
        pushq   %rbx
        call    ffact
        popq    %rbx
        mulq    %rbx
L1:      ret
```

ret to retm
Return to main or calling program
28 March 2024



Recursive Function

```
/* Recursive function ffact */
int ffact( int n)
{
    if (n == 1)
        return 1;
    else
        return (n*ffact(n-1));
}
```

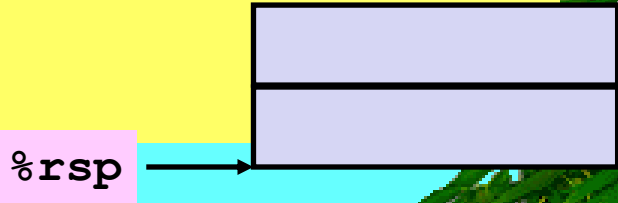
Register	Use(s)	Type
%rax	n	Argument
%rax	Return value	Return value

```
.global  _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:    movq    $nfact, %rdi
         movq    %rax, (%rdi)
         movq    $60, %rax
         xorq    %rdi, %rdi
         syscall
ffact:   cmpq    $1, %rbx
         je      L1
         decq    %rbx
         pushq   %rbx
         call    ffact
rets:    popq    %rbx
         mulq    %rbx
L1:      ret
```

%rsp points to Top of the Stack

28 March 2024

%rax= 8 * 1 * 2 * 3 * 4 * 5 * 6 * 7
= 40320




```
/* Recursive function ffact */  
int ffact( int n)  
{  
    if (n == 1)  
        return 1;  
    else  
        return (n*ffact(n-1));  
}
```

Register	Use(s)	Type
<code>%rax</code>	<i>n</i>	<i>Argument</i>
<code>%rax</code>	<i>Return value</i>	<i>Return value</i>

```
%rax = %rbx = n = 8
```

28 March 2024

The diagram illustrates the state of a program's stack frame and registers during a function call. The assembly code on the left shows the function's logic, while the stack frame on the right shows the return addresses for each call level.

Assembly Code:

```

.global _start
.data
n:      .quad 0x0000000000000008
nfact:  .quad 0x0000000000000000
.text
_start:
    movq    n, %rbx
    movq    %rbx,%rax
    call    ffact
retm:    movq    $nfact, %rdi
         movq    %rax, (%rdi)
         movq    $60, %rax
         xorq    %rdi, %rdi
         syscall
ffact:   cmpq    $1, %rbx
         je      L1
         decq    %rbx
         pushq   %rbx
         call    ffact
rets:    popq    %rbx
         mulq    %rbx
L1:      ret
  
```

Stack Frame:

The stack frame shows the return addresses for each call level, with the return address for the current call being updated by the `rets` instruction. The return address for the initial call is `%rsp`.

Call Level	Return Address
Initial Call	<code>%rsp</code>
Call 1	<code>%rbx=1 (n-7)</code>
Call 2	<code>%rbx=2 (n-6)</code>
Call 3	<code>%rbx=3 (n-5)</code>
Call 4	<code>%rbx=4 (n-4)</code>
Call 5	<code>%rbx=5 (n-3)</code>
Call 6	<code>%rbx=6 (n-2)</code>
Call 7	<code>%rbx=7 (n-1)</code>

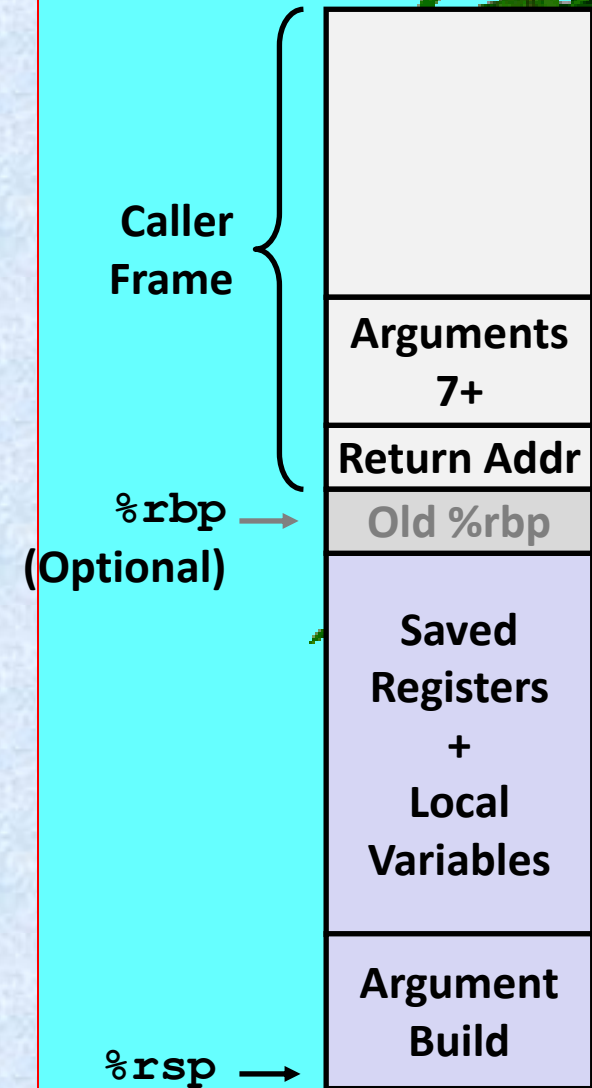
The diagram shows the return address for the current call being updated by the `rets` instruction, which is indicated by a red arrow pointing from the `rets` instruction to the return address field.

`%rax = 8 * 1 * 2 * 3 * 4 * 5 * 6 * 7`
= 40320

	rets address
✓	%rbx=1 (n-7)
	rets address
✓	%rbx=2 (n-6)
	rets address
✓	%rbx=3 (n-5)
	rets address
✓	%rbx=4 (n-4)
	rets address
✓	%rbx=5 (n-3)
	rets address
✓	%rbx=6 (n-2)
	rets address
✓	%rbx=7 (n-1)
	retm address

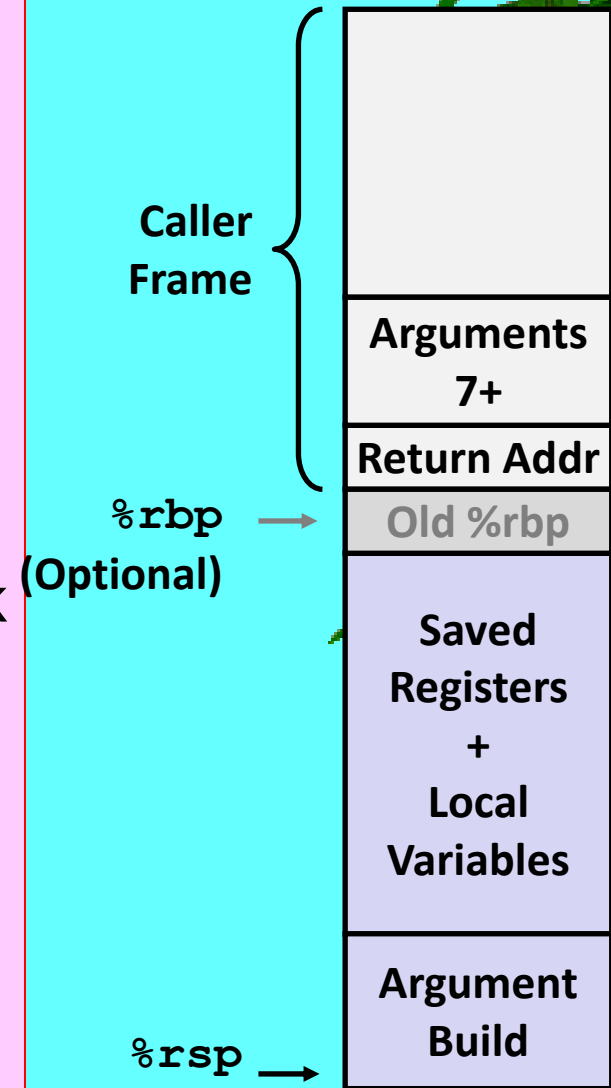
Observations About Recursion

- * Handled Without Special Consideration
 - Stack frames mean that each function call has private storage
 - * Saved registers & local variables
 - * Saved return pointer
 - Register saving conventions prevent one function call from corrupting another's data
 - * Unless the C code explicitly does so (e.g., buffer overflow)
 - Stack discipline follows call / return pattern
 - * If P calls Q, then Q returns before P
 - * Last-In, First-Out
- * Also works for mutual recursion
 - P calls Q; Q calls P



x86-64 Procedure Summary

- ★ Important Points
 - Stack is the right data structure for procedure call / return
 - ★ If P calls Q, then Q returns before P
- ★ Recursion (& mutual recursion) handled by normal calling conventions
 - Can safely store values in local stack frame and in callee-saved registers
 - Put function arguments at top of stack
 - Result return in **%rax**
- ★ Pointers are addresses of values
 - **On stack or global**



SOC 2040

SYSTEM PROGRAMMING

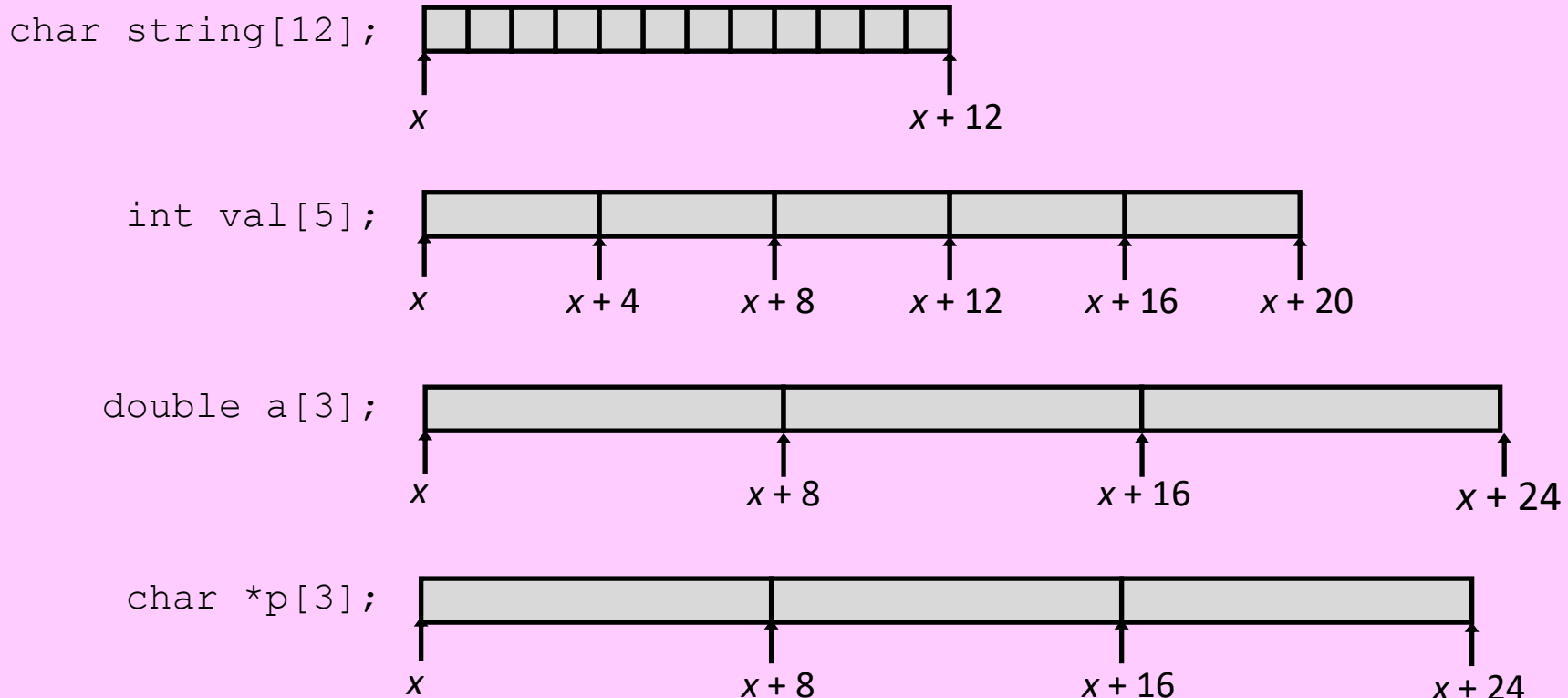
ARRAYS, MULTIDIMENSIONAL ARRAYS & STRUCTURE REPRESENTATION

Array Allocation

* Basic Principle

$T \ A[L];$

- Array of data type T and length L
- Contiguously allocated region of $L * \text{sizeof}(T)$ bytes in memory

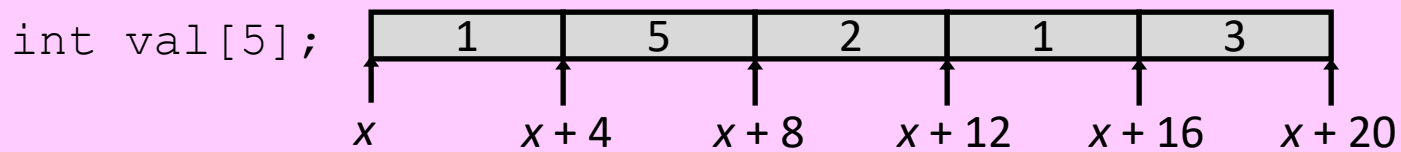


Array Access

* Basic Principle

T $A[L]$;

- Array of data type T and length L
- Identifier A can be used as a pointer to array element 0: Type T^*



* Reference	Type	Value
<code>val[4]</code>	<code>int</code>	3
<code>val</code>	<code>int *</code>	x
<code>val+1</code>	<code>int *</code>	$x + 4$
<code>&val[2]</code>	<code>int *</code>	$x + 8$
<code>val[5]</code>	<code>int</code>	??
<code>*(val+1)</code>	<code>int</code>	5
<code>val + i</code>	<code>int *</code>	$x + 4 i$

Multidimensional (Nested) Arrays

* Declaration

$T \ A[R][C];$

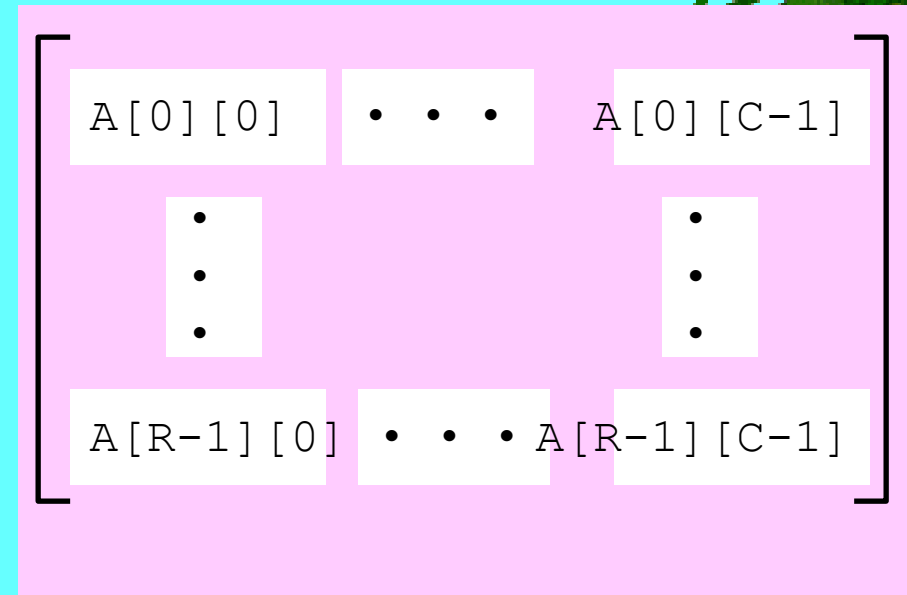
- 2D array of data type T
- R rows, C columns
- Type T element requires K bytes

* Array Size

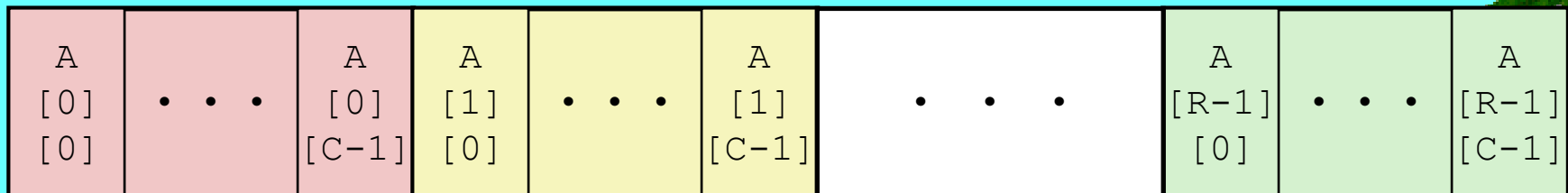
- $R * C * K$ bytes

* Arrangement

- Row-Major Ordering



`int A[R][C];`

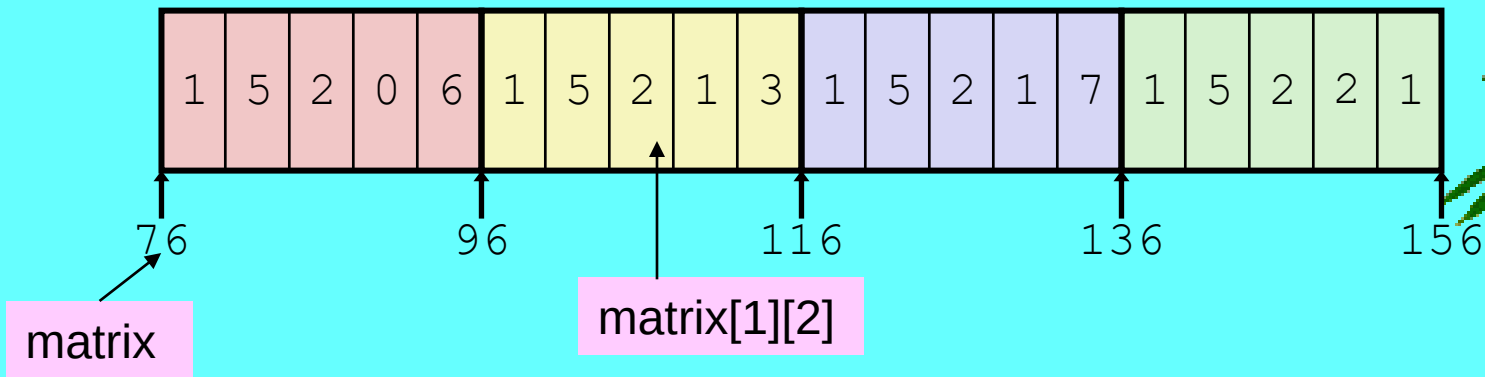


$4 * R * C$ Bytes

Nested Array Example

```
int matrix[4][5]=  
    {{1, 5, 2, 0, 6},  
     {1, 5, 2, 1, 3 },  
     {1, 5, 2, 1, 7 },  
     {1, 5, 2, 2, 1 }};
```

```
int matrix[4];
```



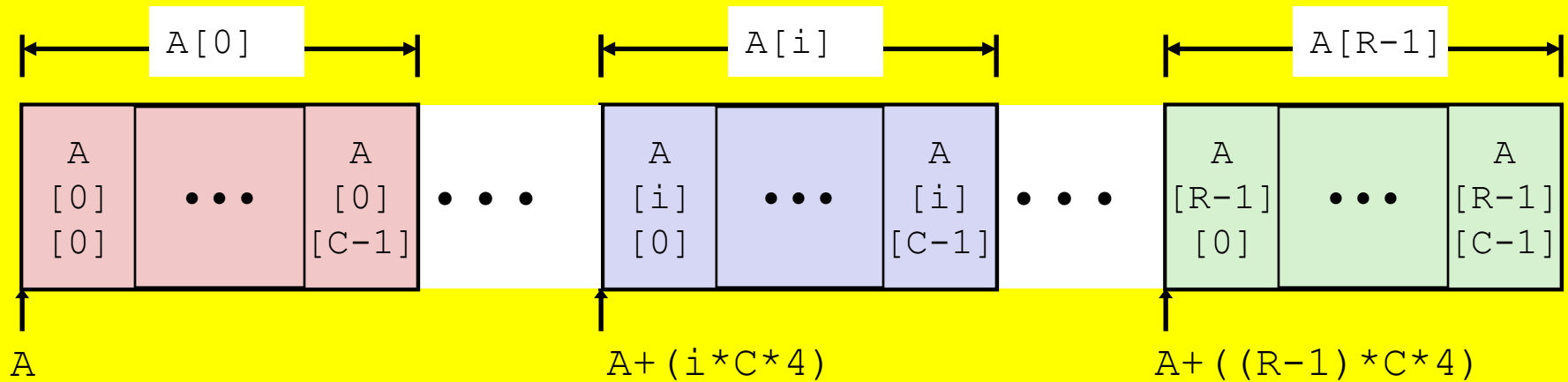
- * **MATRIX OF 4 ROWS and 5 COLUMNS**
- * “Row-Major” ordering of all elements in memory

Nested Array Row Access

* Row Vectors

- $A[i]$ is array of C elements
- Each element of type T requires K bytes
- Starting address $A + i * (C * K)$

```
int A[R][C];
```

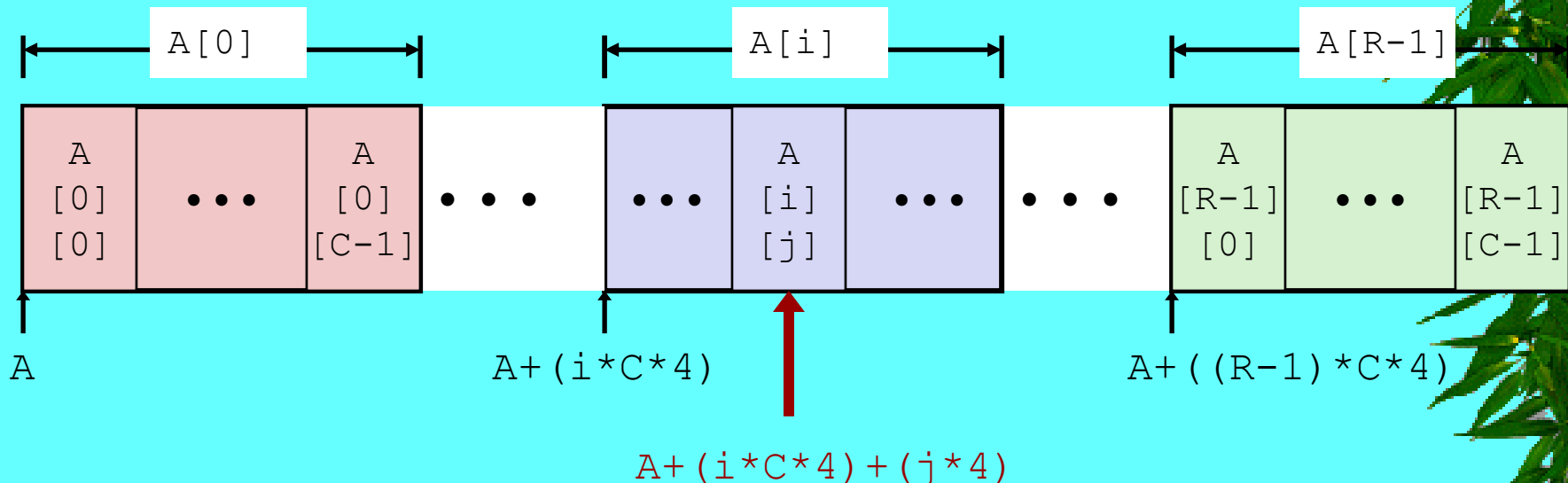


Nested Array Element Access

★ Array Elements

- $A[i][j]$ is element of type T , which requires K bytes
- Address $A + i * (C * K) + j * K = A + (i * C + j) * K$

```
int A[R][C];
```



16 X 16 Matrix Access

- *Array Elements*
- $\text{Address } A + i * (C * K) + j * K$
- $C = 16, K = 4$

```
/* Get element a[i][j] */  
int fix_ele(fix_matrix a, size_t i, size_t j) {  
    return a[i][j];  
}
```

```
# a in %rdi, i in %rsi, j in %rdx  
salq    $6, %rsi           # 64*i  
addq    %rsi, %rdi          # a + 64*i  
movq    (%rdi,%rdx,4), %rax # M[a + 64*i + 4*j]  
ret
```


n X n Matrix Access

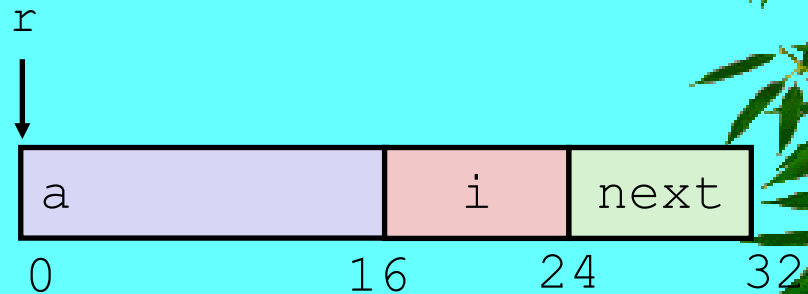
- *Array Elements*
- $Address\ A + i * (C * K) + j * K$
- $C = n, K = 4$
- Must perform integer multiplication

```
/* Get element a[i][j] */  
int var_ele(size_t n, int a[n][n], size_t i, size_t j)  
{  
    return a[i][j];  
}
```

```
# n in %rdi, a in %rsi, i in %rdx, j in %rcx  
imulq    %rdx, %rdi          # n*i  
leaq     (%rsi,%rdi,4), %rax  # a + 4*n*i  
movq     (%rax,%rcx,4), %rax  # a + 4*n*i + 4*j  
ret
```

Structure Representation

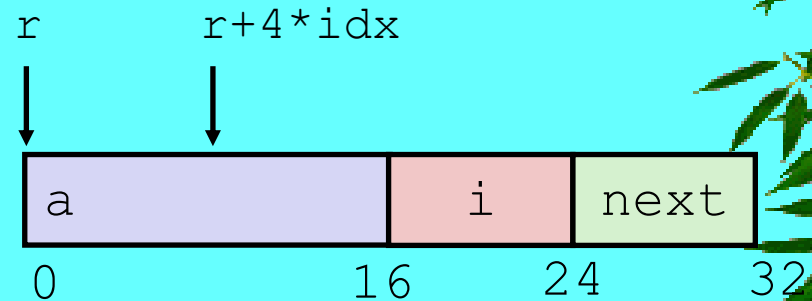
```
struct rec {  
    int a[4];  
    size_t i;  
    struct rec *next;  
};
```



- * Structure represented as block of memory
 - Big enough to hold all of the fields
- * Fields ordered according to declaration
 - Even if another ordering could yield a more compact representation
- * Compiler determines overall size + positions of fields
 - Machine-level program has no understanding of the structures in the source code

Generating Pointer to Structure Member

```
struct rec {  
    int a[4];  
    size_t i;  
    struct rec *next;  
};
```



* Generating Pointer to Array Element

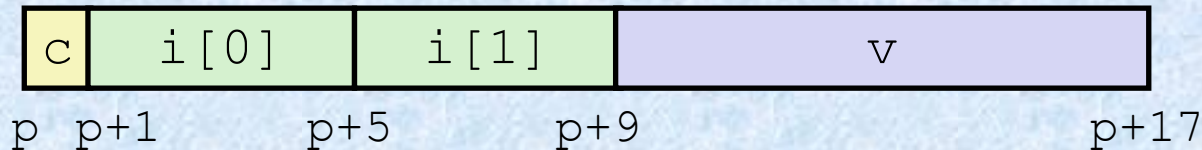
- Offset of each structure member determined at compile time
- Compute as $r + 4 * idx$

```
int *get_ap  
(struct rec *r, size_t idx)  
{  
    return &r->a[idx];  
}
```

```
# r in %rdi, idx in %rsi  
leaq (%rdi,%rsi,4), %rax  
ret
```

Structures & Alignment

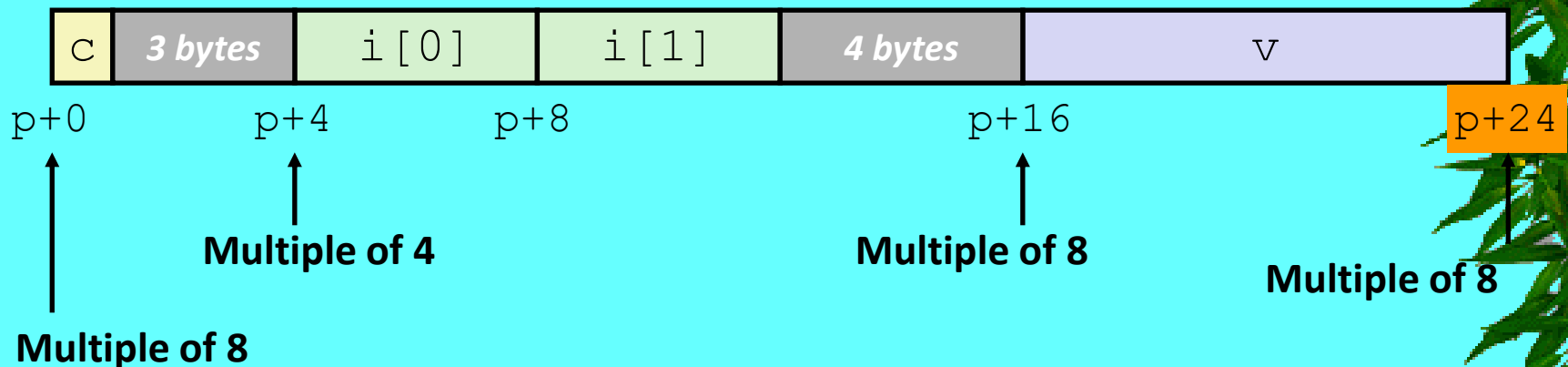
★ Unaligned Data



```
struct S1 {  
    char c;  
    int i[2];  
    double v;  
} *p;
```

★ Aligned Data

- Primitive data type requires K bytes
- Address must be multiple of K



Alignment Principles

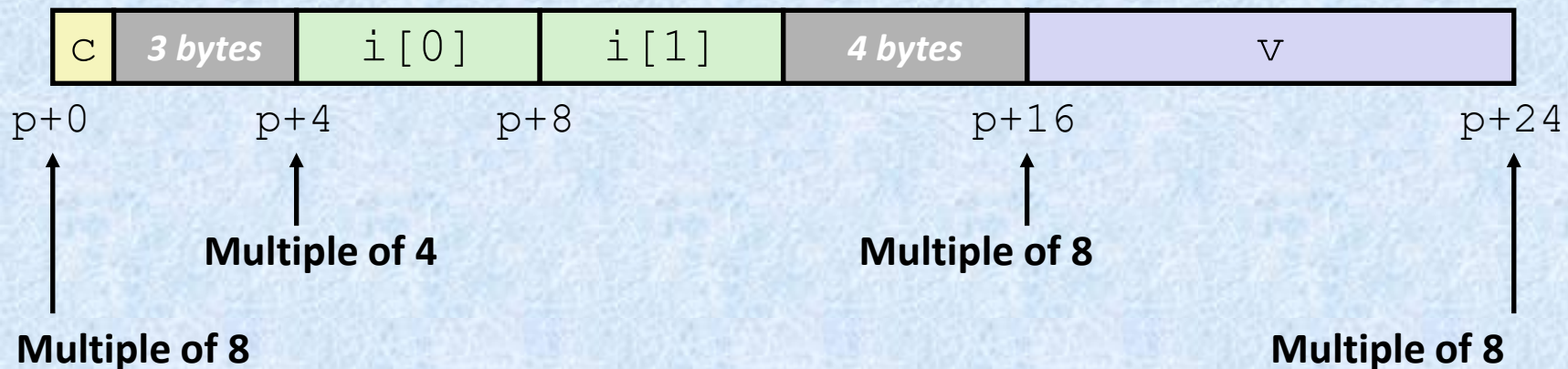
- ★ Aligned Data
 - Primitive data type requires K bytes
 - Address must be multiple of K
 - Required on some machines; advised on x86-64
- ★ Motivation for Aligning Data
 - Memory accessed by (aligned) chunks of 4 or 8 bytes (system dependent)
 - ★ Inefficient to load or store datum that spans quad word boundaries
 - ★ Virtual memory trickier when datum spans 2 pages
- ★ Compiler
 - Inserts gaps in structure to ensure correct alignment of fields

Specific Cases of Alignment (x86-64)

- ★ 1 byte: **char**, ...
 - no restrictions on address
- ★ 2 bytes: **short**, ...
 - lowest 1 bit of address must be 0_2
- ★ 4 bytes: **int**, **float**, ...
 - lowest 2 bits of address must be 00_2
- ★ 8 bytes: **double**, `long`, **char ***, ...
 - lowest 3 bits of address must be 000_2
- ★ 16 bytes: **long double** (GCC on Linux)
 - lowest 4 bits of address must be 0000_2

Satisfying Alignment with Structures

- ★ Within structure:
 - Must satisfy each element's alignment requirement
- ★ Overall structure placement
 - Each structure has alignment requirement **K**
 - ★ **K** = Largest alignment of any element
 - Initial address & structure length must be multiples of **K**
- ★ Example:
 - K** = 8, due to **double** element

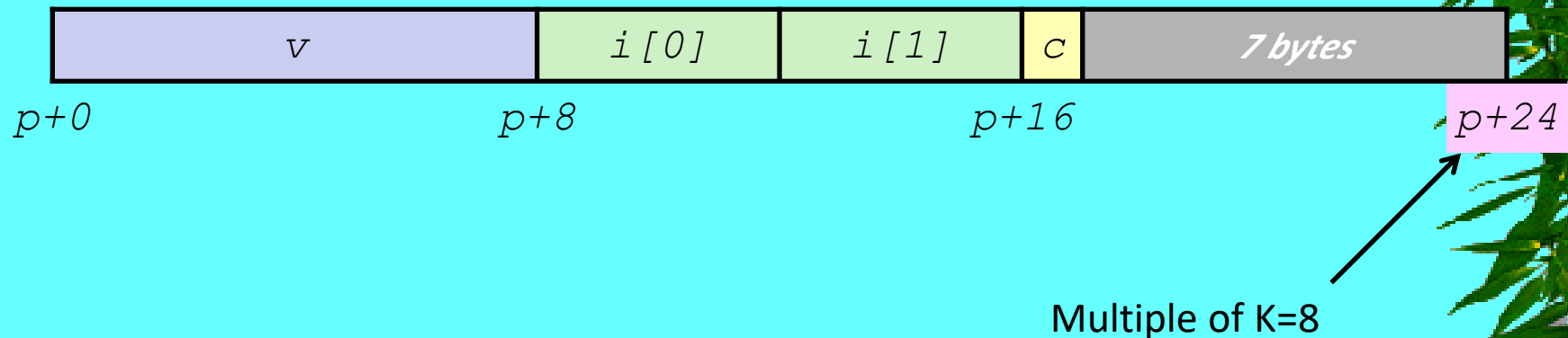


```
struct S1 {  
    char c;  
    int i[2];  
    double v;  
} *p;
```


Meeting Overall Alignment Requirement

- ★ For largest alignment requirement K
- ★ Overall structure must be multiple of K

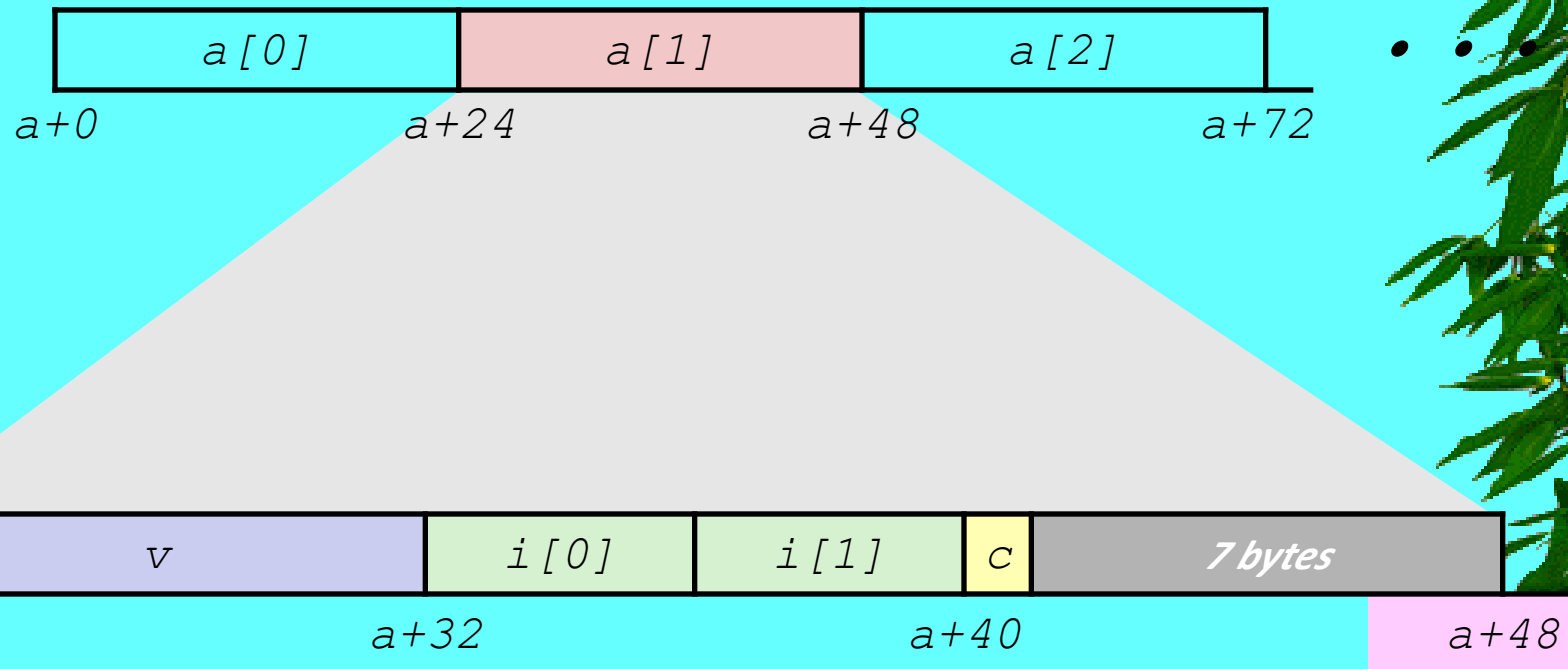
```
struct S2 {  
    double v;  
    int i[2];  
    char c;  
} *p;
```



Arrays of Structures

- ★ Overall structure length multiple of K
- ★ Satisfy alignment requirement for every element

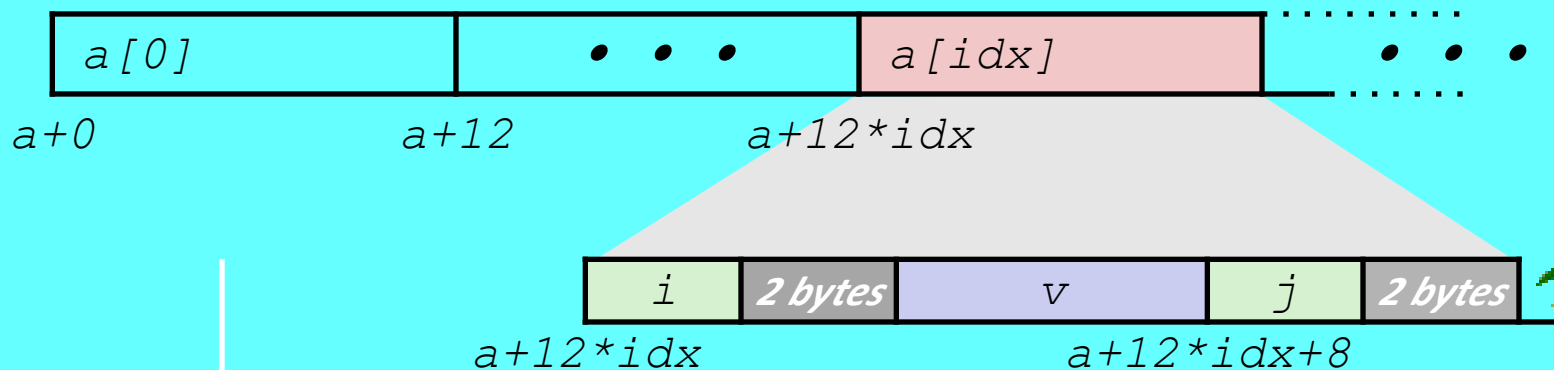
```
struct S2 {  
    double v;  
    int i[2];  
    char c;  
} a[10];
```



Accessing Array Elements

- ★ Compute array offset $12 \cdot \text{idx}$
 - `sizeof(S3)`, including alignment spacers
- ★ Element `j` is at offset 8 within structure
- ★ Assembler gives offset `a+8`
 - Resolved during linking

```
struct S3 {  
    short i;  
    float v;  
    short j;  
} a[10];
```



```
short get_j(int idx)  
{  
    return a[idx].j;  
}
```

```
# %rdi = idx  
leaq (%rdi,%rdi,2),%rax # 3*idx  
movw a+8(,%rax,4),%ax
```

Better Ordering resulting in a saving of 4 bytes of memory space

```
struct S3 {  
    float v;  
    short i;  
    short j;  
} a[10];
```

```
struct S3 {  
    short i;  
    short j;  
    float v;  
} a[10];
```

Saving Space

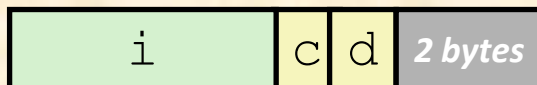
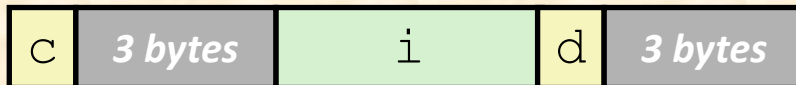
- ★ Put large data types first

```
struct S4 {  
    char c;  
    int i;  
    char d;  
} *p;
```



```
struct S5 {  
    int i;  
    char c;  
    char d;  
} *p;
```

- ★ Effect (K=4)



SOC 2040

SYSTEM PROGRAMMING

FLOATING POINT INSTRUCTIONS

Background

❑ History

❖ x87 FP & MMX SIMD

- **Multi-Media Extensions-Single Instruction Multiple Data**
- MMX Registers 64-bit

❖ SSE FP

- ★ **Streaming SIMD Extensions – SSE2, SSE3, SSE4**
- ★ Special case use of vector instructions
- ★ XMM0 to XMM15 registers - 128-bit

❖ AVX FP

★ **Advanced Vector Extensions – AVX, AVX2**

- ★ Newest version, Similar to SSE
- ★ YMM0 TO YMM15 registers – 256-bit

★ **AVX-512 with 512-bit registers : ZMM0 TO ZMM15**

- ★ Intel Advanced Vector Extensions 512 (Intel AVX-512) is a set of new instructions that can accelerate performance for workloads and usages such as scientific simulations, financial analytics, artificial intelligence (AI)/deep learning, 3D modeling and analysis, image and audio/video processing, cryptography and data compression.

AVX-512 register scheme as extension from the AVX (YMM0-YMM15) and SSE (XMM0-XMM15) registers					
511	256	255	128	127	0
ZMM0		YMM0		XMM0	
ZMM1		YMM1		XMM1	
ZMM2		YMM2		XMM2	
ZMM3		YMM3		XMM3	
ZMM4		YMM4		XMM4	
ZMM5		YMM5		XMM5	
ZMM6		YMM6		XMM6	
ZMM7		YMM7		XMM7	
ZMM8		YMM8		XMM8	
ZMM9		YMM9		XMM9	
ZMM10		YMM10		XMM10	
ZMM11		YMM11		XMM11	
ZMM12		YMM12		XMM12	
ZMM13		YMM13		XMM13	
ZMM14		YMM14		XMM14	
ZMM15		YMM15		XMM15	
ZMM16		YMM16		XMM16	
ZMM17		YMM17		XMM17	
ZMM18		YMM18		XMM18	
ZMM19		YMM19		XMM19	
ZMM20		YMM20		XMM20	
ZMM21		YMM21		XMM21	
ZMM22		YMM22		XMM22	
ZMM23		YMM23		XMM23	
ZMM24		YMM24		XMM24	
ZMM25		YMM25		XMM25	
ZMM26		YMM26		XMM26	
ZMM27		YMM27		XMM27	
ZMM28		YMM28		XMM28	
ZMM29		YMM29		XMM29	
ZMM30		YMM30		XMM30	
ZMM31		YMM31		XMM31	

Programming with SSE

XMM Registers

■ 16 total, each 16 bytes

■ 16 single-byte integers



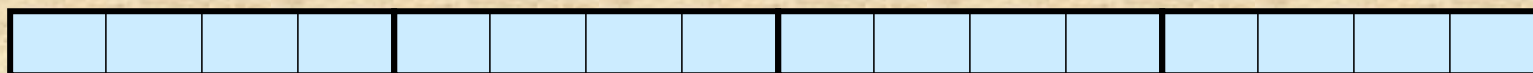
■ 8 16-bit integers



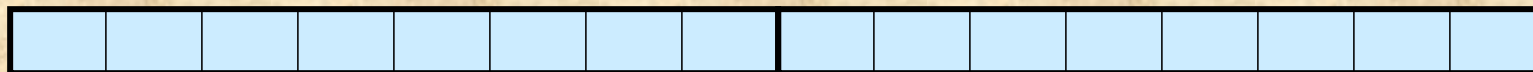
■ 4 32-bit integers



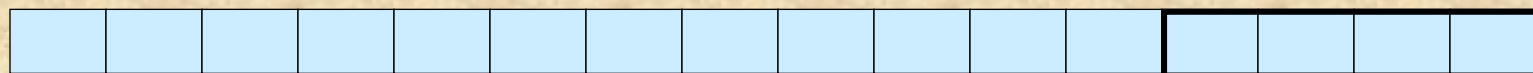
■ 4 single-precision floats



■ 2 double-precision floats



■ 1 single-precision float

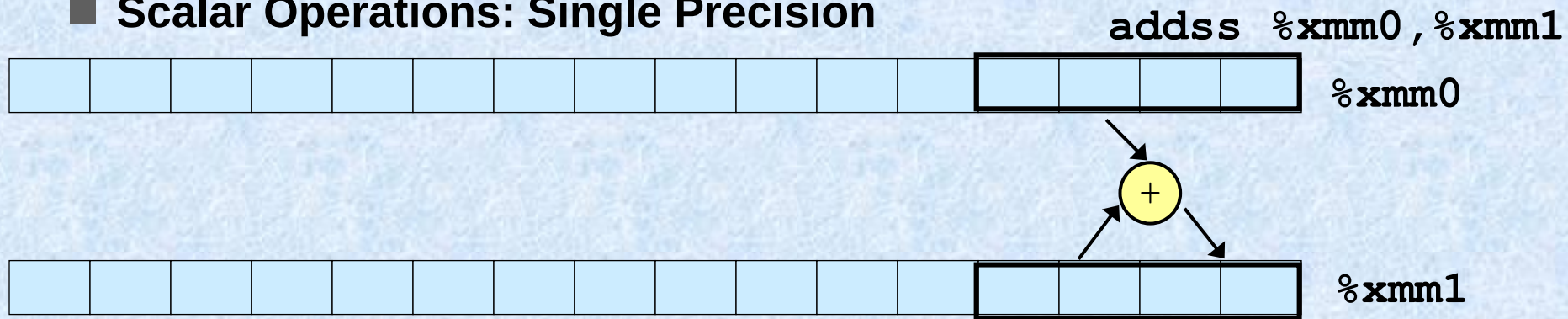


■ 1 double-precision float

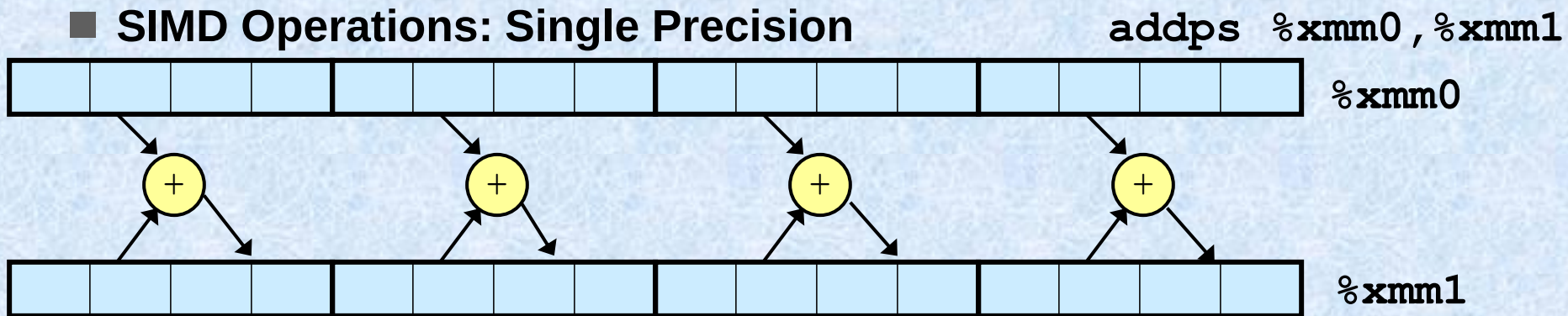


Scalar & SIMD Operations

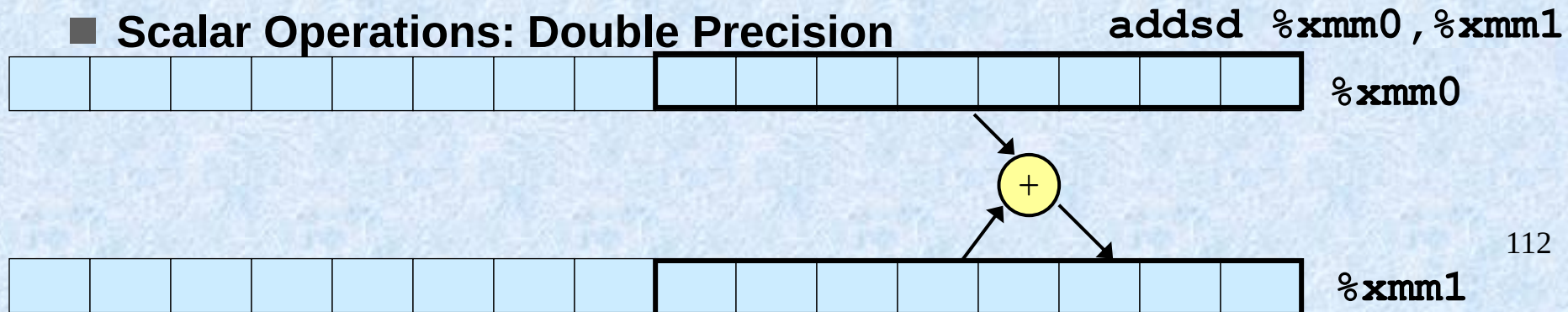
■ Scalar Operations: Single Precision



■ SIMD Operations: Single Precision



■ Scalar Operations: Double Precision



FP Basics

- ★ Arguments passed in `%xmm0`, `%xmm1`, ...
- ★ Result returned in `%xmm0`
- ★ All XMM registers caller-saved

```
float fadd(float x, float y)
{
    return x + y;
}
```

```
double dadd(double x, double y)
{
    return x + y;
}
```

```
# x in %xmm0, y in %xmm1
addss    %xmm1, %xmm0
ret
```

```
# x in %xmm0, y in %xmm1
addsd    %xmm1, %xmm0
ret
```

FP Memory Referencing

- ★ Integer (and pointer) arguments passed in regular registers
- ★ FP values passed in XMM registers
- ★ Different mov instructions to move between XMM registers, and between memory and XMM registers

```
double dincr(double *p, double v)
{
    double x = *p;
    *p = x + v;
    return x;
}
```

```
# p in %rdi, v in %xmm0
movapd  %xmm0, %xmm1    # Copy v
movsd   (%rdi), %xmm0    # x = *p
addsd   %xmm0, %xmm1    # t = x + v
movsd   %xmm1, (%rdi)    # *p = t
ret
```

Summary

★ Arrays

- Elements packed into contiguous region of memory
- Use index arithmetic to locate individual elements

★ Structures

- Elements packed into single region of memory
- Access using offsets determined by compiler
- Possible require internal and external padding to ensure alignment

★ Combinations

- Can nest structure and array code arbitrarily

★ Floating Point

- Data held and operated on in XMM, YMM and ZMM registers

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapter 3 of Text Book

➤ Computer Systems : A Programmer's Perspective

- Randal E. Bryant and David R. O'Hallaron
3rd Edition, Global Edition Prentice Hall 2016
ISBN 10:1-292-10176-8**

SOC 2040

SYSTEM PROGRAMMING

**END OF WEEK7
LECTURE**

**MACHINE-LEVEL
PROGRAMMING II**

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

